

Team 3

Team Members: Shah Tirth, Rishank Soni, Anirudh Mittal, Shail Bhargavkumar Joshi,
Arjun Anand Mallya, Krish Patel, Kshitij Agarwal, Shriniket Behra, Moksh Jindal,
Mayank Kumbhat

Document 1: RISCv Modules [Codes]

RV32I46F_5SP.v

```
`include "modules/Program_Counter.v"
`include "modules/PC_Controller.v"
`include "modules/PC_Plus_4.v"
`include "modules/Instruction_Memory.v"
`include "modules/Instruction_Decoder.v"
`include "modules/Immediate_Generator.v"
`include "modules/Control_Unit.v"
`include "modules/Register_File.v"
`include "modules/Data_Memory.v"
`include "modules/ALU_Controller.v"
`include "modules/ALU.v"
`include "modules/Branch_Logic.v"
`include "modules/Byte_Enable_Logic.v"
`include "modules/CSR_File.v"
`include "modules/Exception_Detector.v"
`include "modules/Trap_Controller.v"

`include "modules/IF_ID_Register.v"
`include "modules/ID_EX_Register.v"
`include "modules/EX_MEM_Register.v"
`include "modules/MEM_WB_Register.v"

`include "modules/Hazard_Unit.v"
`include "modules/Forward_Unit.v"
`include "modules/Branch_Predictor.v"

`include "modules/headers/alu_src_select.vh"
`include "modules/headers/rf_wd_select.vh"

module RV32I46F5SP #(
    parameter XLEN = 32
)(
    input clk,
    input reset,

    output wire [31:0] retire_instruction
);
    // Program Counter and PC Plus 4
    wire [XLEN-1:0] pc;
    wire [XLEN-1:0] pc_plus_4_signal;
    wire [XLEN-1:0] next_pc;
```

```

// Instruction Memory and Debug Interface
wire [31:0] im_instruction;
wire [31:0] dbg_instruction = 32'b00000001011110110000110000110011; //add x24 = x22 + x23 = FFFF_FFBC +
ABAD_BB02 = ABADBABE
reg [31:0] instruction;
wire [XLEN-1:0] IF_imm;
wire [6:0] IF_opcode;

assign IF_imm = {{20{instruction[31]}}, instruction[7], instruction[30:25], instruction[11:8], 1'b0};
assign IF_opcode = (instruction[6:0]);

// Instruction Decoder
wire [6:0] opcode;
wire [2:0] funct3;
wire [6:0] funct7;
wire [4:0] rs1;
wire [4:0] rs2;
        wire [4:0] rd;
wire [19:0] raw_imm;

// Immediate Generator
wire [XLEN-1:0] imm;

// Control Unit
wire pc_stall;
wire jump;
        wire branch;
        wire [1:0] alu_src_A_select;
        wire [2:0] alu_src_B_select;
wire memory_read;
        wire memory_write;
        wire register_file_write;
        wire [2:0] register_file_write_data_select;
wire cu_csr_write_enable;

// Branch Logic and Branch Predictor
wire branch_taken;
wire [XLEN-1:0] branch_target;
wire [XLEN-1:0] branch_target_actual;

// Register File
reg [XLEN-1:0] register_file_write_data;

// Register File
wire [XLEN-1:0] read_data1;
wire [XLEN-1:0] read_data2;

// ALU Controller
wire [3:0] alu_op;

// ALUsrcA, srcB MUX
reg [XLEN-1:0] src_A;
reg [XLEN-1:0] src_B;

```

```

// ALU
wire [XLEN-1:0] alu_result;
wire alu_zero;

// Data Memory and Byte Enable Logic
wire [XLEN-1:0] data_memory_read_data;
    wire [XLEN-1:0] byte_enable_logic_register_file_write_data;
wire [XLEN-1:0] data_memory_write_data;
wire [3:0] write_mask;

// CSR File
wire csr_write_enable;
reg [4:0] csr_rs1;
reg [11:0] csr_read_address;
reg [11:0] csr_write_address;
reg [XLEN-1:0] csr_write_data;
wire [XLEN-1:0] csr_read_out;
wire illegal_csr;

// Exception_Detector
wire trapped;
wire [2:0] trap_status;

// Trap Controller
wire trap_done;
wire debug_mode;
wire tc_csr_write_enable;
wire [XLEN-1:0] trap_target;
wire [11:0] csr_trap_address;
wire [XLEN-1:0] csr_trap_write_data;
wire pth_done_flush;

// IF_ID_Register
wire [XLEN-1:0] ID_pc;
wire [XLEN-1:0] ID_pc_plus_4;
wire [31:0] ID_instruction;
wire ID_branch_estimation;

// ID_EX_Register
wire [XLEN-1:0] EX_pc;
wire [XLEN-1:0] EX_pc_plus_4;
wire EX_branch_estimation;
wire [31:0] EX_instruction;

// EX_MEM_Register
wire EX_jump;
wire EX_memory_read;
wire EX_memory_write;
wire [2:0] EX_register_file_write_data_select;
wire EX_register_write_enable;
wire EX_csr_write_enable;
wire EX_branch;
wire [1:0] EX_alu_src_A_select;

```

```

wire [2:0] EX_alu_src_B_select;
wire [6:0] EX_opcode;
wire [2:0] EX_funct3;
wire [6:0] EX_funct7;
wire [4:0] EX_rd;
wire [19:0] EX_raw_imm;
wire [XLEN-1:0] EX_read_data1;
wire [XLEN-1:0] EX_read_data2;
wire [4:0] EX_rs1;
wire [4:0] EX_rs2;
wire [XLEN-1:0] EX_imm;
wire [XLEN-1:0] EX_csr_read_data;

wire [XLEN-1:0] MEM_pc;
wire [XLEN-1:0] MEM_pc_plus_4;
wire [31:0] MEM_instruction;

wire MEM_memory_read;
wire MEM_memory_write;
wire [2:0] MEM_register_file_write_data_select;
wire MEM_register_write_enable;
wire MEM_csr_write_enable;
wire [6:0] MEM_opcode;
wire [2:0] MEM_funct3;
wire [4:0] MEM_rs1;
wire [4:0] MEM_rd;
wire [XLEN-1:0] MEM_read_data2;
wire [XLEN-1:0] MEM_imm;
wire [19:0] MEM_raw_imm;
wire [XLEN-1:0] MEM_csr_read_data;
wire [XLEN-1:0] MEM_alu_result;

wire [XLEN-1:0] WB_pc;
wire [XLEN-1:0] WB_pc_plus_4;
wire [31:0] WB_instruction;
wire [6:0] WB_opcode;

// MEM_WB_Register
wire MEM_WB_flush;
wire [2:0] WB_register_file_write_data_select;
wire [XLEN-1:0] WB_imm;
wire [19:0] WB_raw_imm;
wire [XLEN-1:0] WB_csr_read_data;
wire [XLEN-1:0] WB_alu_result;
wire WB_register_write_enable;
wire WB_csr_write_enable;
wire [4:0] WB_rs1;
wire [4:0] WB_rd;

wire [XLEN-1:0] WB_byte_enable_logic_register_file_write_data;

// Hazard Unit
wire IF_ID_flush;
wire ID_EX_flush;

```

```

wire IF_ID_stall;
wire ID_EX_stall;
wire EX_MEM_stall;
wire MEM_WB_stall;
wire csr_hazard_mem;
wire csr_hazard_wb;

// Forward Unit
wire [1:0] hazard_mem;
wire [1:0] hazard_wb;
wire [XLEN-1:0] csr_forward_data;
wire [XLEN-1:0] alu_forward_source_data_a;
wire [XLEN-1:0] alu_forward_source_data_b;
wire [1:0] alu_forward_source_select_a;
wire [1:0] alu_forward_source_select_b;
reg [XLEN-1:0] alu_normal_source_a;
reg [XLEN-1:0] alu_normal_source_b;

reg [XLEN-1:0] retired_alu_result;
reg [XLEN-1:0] retired_csr_read_data;

// Branch Predictor
wire branch_estimation;

wire [31:0] writeback_instruction = WB_instruction;
assign retire_instruction = writeback_instruction;

wire csr_write_enable_source;
assign csr_write_enable_source = tc_csr_write_enable ? tc_csr_write_enable : WB_csr_write_enable;

ProgramCounter program_counter (
    .clk(clk),
    .reset(reset),
    .next_pc(next_pc),
    .pc(pc)
);

PCController pc_controller (
    .jump(EX_jump),
    .branch_estimation(branch_estimation),
    .branch_prediction_miss(branch_prediction_miss),
    .trapped(trapped),
    .pc(pc),
    .jump_target(alu_result),
    .branch_target(branch_target),
    .branch_target_actual(branch_target_actual),
    .trap_target(trap_target),
    .pc_stall(pc_stall),
    .next_pc(next_pc)
);

PCPlus4 pc_plus_4 (
    .pc(pc),
    .pc_plus_4(pc_plus_4_signal)

```

);

```
InstructionMemory instruction_memory (  
    .pc(pc),  
    .instruction(im_instruction)  
);
```

```
InstructionDecoder instruction_decoder (  
    .instruction(ID_instruction),  
    .opcode(opcode),  
    .funct3(funct3),  
    .funct7(funct7),  
    .rs1(rs1),  
    .rs2(rs2),  
    .rd(rd),  
    .raw_imm(raw_imm)  
);
```

```
ImmediateGenerator immediate_generator (  
    .raw_imm(raw_imm),  
    .opcode(opcode),  
    .imm(imm)  
);
```

```
ControlUnit control_unit (  
    .write_done(1'b1),  
    .opcode(opcode),  
    .funct3(funct3),  
    .trap_done(trap_done),  
    .csr_ready(csr_ready),  
  
    .pc_stall(pc_stall),  
    .jump(jump),  
    .branch(branch),  
    .alu_src_A_select(alu_src_A_select),  
    .alu_src_B_select(alu_src_B_select),  
    .register_file_write(register_file_write),  
    .register_file_write_data_select(register_file_write_data_select),  
    .memory_read(memory_read),  
    .memory_write(memory_write),  
    .csr_write_enable(cu_csr_write_enable)  
);
```

```
RegisterFile register_file (  
    .clk(clk),  
    .read_reg1(rs1),  
    .read_reg2(rs2),  
    .write_reg(WB_rd),  
    .write_data(register_file_write_data),  
    .write_enable(WB_register_write_enable),  
  
    .read_data1(read_data1),  
    .read_data2(read_data2)  
);
```

```
DataMemory data_memory (
    .clk(clk),
    .write_enable(MEM_memory_write),
    .address(MEM_alu_result[9:0]),
    .write_data(data_memory_write_data),
    .write_mask(write_mask),

    .read_data(data_memory_read_data)
);
```

```
ALUController alu_controller (
    .opcode(EX_opcode),
    .funct3(EX_funct3),
    .funct7_5(EX_funct7[5]),
    .imm_10(EX_imm[10]),

    .alu_op(alu_op)
);
```

```
ALU alu (
    .src_A(src_A),
    .src_B(src_B),
    .alu_op(alu_op),

    .alu_result(alu_result),
    .alu_zero(alu_zero)
);
```

```
BranchLogic branch_logic (
    .branch(EX_branch),
    .branch_estimation(EX_branch_estimation),
    .funct3(EX_funct3),
    .alu_zero(alu_zero),
    .pc(EX_pc),
    .imm(EX_imm),

    .branch_taken(branch_taken),
    .branch_prediction_miss(branch_prediction_miss),
    .branch_target_actual(branch_target_actual)
);
```

```
ByteEnableLogic byte_enable_logic (
    .memory_read(MEM_memory_read),
    .memory_write(MEM_memory_write),
    .funct3(MEM_funct3),
    .register_file_read_data(MEM_read_data2),
    .data_memory_read_data(data_memory_read_data),
    .address(MEM_alu_result[1:0]),

    .register_file_write_data(byte_enable_logic_register_file_write_data),
    .data_memory_write_data(data_memory_write_data),
    .write_mask(write_mask)
);
```

```

CSRFile #(.XLEN(XLEN)) csr_file (
    .clk(clk),
    .reset(reset),
    .trapped(trapped),
    .csr_write_enable(csr_write_enable_source),
    .csr_read_address(csr_read_address),
    .csr_write_address(csr_write_address),
    .csr_write_data(csr_write_data),

    .csr_read_out(csr_read_out),
    .csr_ready(csr_ready)
);

```

```

ExceptionDetector exception_detector (
    .clk(clk),
    .reset(reset),
    .ID_opcode(opcode),
    .ID_funct3(funct3),
    .EX_opcode(EX_opcode),
    .EX_funct3(EX_funct3),
    .MEM_opcode(MEM_opcode),
    .MEM_funct3(MEM_funct3),
    .raw_imm(raw_imm[11:0]),
    .EX_raw_imm(EX_raw_imm[11:0]),
    .csr_write_enable(cu_csr_write_enable),
    .alu_result(alu_result[1:0]), // for jump_target_lsbs and data_memory_address_lsbs
    .MEM_alu_result(MEM_alu_result[1:0]),
    .branch_target_lsbs(branch_target[1:0]),
    .branch_estimation(branch_estimation),

    .trapped(trapped),
    .trap_status(trap_status)
);

```

```

TrapController #(.XLEN(XLEN)) trap_controller (
    .clk(clk),
    .reset(reset),
    .trap_status(trap_status),
    .ID_pc(ID_pc),
    .EX_pc(EX_pc),
    .MEM_pc(MEM_pc),
    .WB_pc(WB_pc),
    .csr_read_data(csr_read_out),

    .debug_mode(debug_mode),
    .trap_target(trap_target),
    .trap_done(trap_done),
    .misaligned_instruction_flush(misaligned_instruction_flush),
    .misaligned_memory_flush(misaligned_memory_flush),
    .pth_done_flush(pth_done_flush),
    .standby_mode(standby_mode),
    .csr_write_enable(tc_csr_write_enable),
    .csr_trap_address(csr_trap_address),

```



```

        .csr_trap_write_data(csr_trap_write_data)
    );

IF_ID_Register #(XLEN(XLEN)) if_id_register (
    .clk(clk),
        .reset(reset),
        .flush(IF_ID_flush),
        .IF_ID_stall(IF_ID_stall),

    // Signals from IF Phase
    .IF_pc(pc),
    .IF_pc_plus_4(pc_plus_4_signal),
    .IF_instruction(instruction),
    .IF_branch_estimation(branch_estimation),

    // Signals to ID_EX_Register and ID Phase
    .ID_pc(ID_pc),
    .ID_pc_plus_4(ID_pc_plus_4),
    .ID_instruction(ID_instruction),
    .ID_branch_estimation(ID_branch_estimation)
);

ID_EX_Register #(XLEN(XLEN)) id_ex_register (
    .clk(clk),
        .reset(reset),
        .flush(ID_EX_flush),
        .ID_EX_stall(ID_EX_stall),

    // Signals from IF_ID_Register
    .ID_pc(ID_pc),
    .ID_pc_plus_4(ID_pc_plus_4),
    .ID_branch_estimation(ID_branch_estimation),
    .ID_instruction(ID_instruction),

    // Signals from ID Phase
    .ID_jump(jump),
    .ID_branch(branch),
    .ID_alu_src_A_select(alu_src_A_select),
    .ID_alu_src_B_select(alu_src_B_select),
    .ID_memory_read(memory_read),
    .ID_memory_write(memory_write),
    .ID_register_file_write_data_select(register_file_write_data_select),
    .ID_register_write_enable(register_file_write),
    .ID_csr_write_enable(cu_csr_write_enable),
    .ID_opcode(opcode),
    .ID_funct3(funct3),
    .ID_funct7(funct7),
    .ID_rd(rd),
    .ID_raw_imm(raw_imm),
    .ID_read_data1(read_data1),
    .ID_read_data2(read_data2),
    .ID_rs1(rs1),
    .ID_rs2(rs2),
    .ID_imm(imm),

```

```

.ID_csr_read_data(csr_read_out),

// Signals to EX_MEM_Register
.EX_pc(EX_pc),
.EX_pc_plus_4(EX_pc_plus_4),
.EX_branch_estimation(EX_branch_estimation),
.EX_instruction(EX_instruction),

.EX_jump(EX_jump),
.EX_branch(EX_branch),
.EX_alu_src_A_select(EX_alu_src_A_select),
.EX_alu_src_B_select(EX_alu_src_B_select),
.EX_memory_read(EX_memory_read),
.EX_memory_write(EX_memory_write),
.EX_register_file_write_data_select(EX_register_file_write_data_select),
.EX_register_write_enable(EX_register_write_enable),
.EX_csr_write_enable(EX_csr_write_enable),
.EX_opcode(EX_opcode),
.EX_funct3(EX_funct3),
.EX_funct7(EX_funct7),
.EX_rd(EX_rd),
.EX_raw_imm(EX_raw_imm),
.EX_read_data1(EX_read_data1),
.EX_read_data2(EX_read_data2),
.EX_rs1(EX_rs1),
.EX_rs2(EX_rs2),
.EX_imm(EX_imm),
.EX_csr_read_data(EX_csr_read_data)
);

EX_MEM_Register #(.XLEN(XLEN)) ex_mem_register (
    .clk(clk),
        .reset(reset),
    .flush(EX_MEM_flush),
    .EX_MEM_stall(EX_MEM_stall),

// Signals from ID_EX_Register
.EX_pc(EX_pc),
.EX_pc_plus_4(EX_pc_plus_4),
.EX_instruction(EX_instruction),

.EX_memory_read(EX_memory_read),
.EX_memory_write(EX_memory_write),
.EX_register_file_write_data_select(EX_register_file_write_data_select),
.EX_register_write_enable(EX_register_write_enable),
.EX_csr_write_enable(EX_csr_write_enable),
.EX_opcode(EX_opcode),
.EX_funct3(EX_funct3),
.EX_rs1(EX_rs1),
.EX_rd(EX_rd),
.EX_raw_imm(EX_raw_imm),
.EX_read_data2(EX_read_data2),
.EX_imm(EX_imm),
.EX_csr_read_data(EX_csr_read_data),

```

```

// Signal from EX Phase
.EX_alu_result(alu_result),

// Signals to MEM_WB_Register
.MEM_pc(MEM_pc),
.MEM_pc_plus_4(MEM_pc_plus_4),
.MEM_instruction(MEM_instruction),
.MEM_memory_read(MEM_memory_read),
.MEM_memory_write(MEM_memory_write),
.MEM_register_file_write_data_select(MEM_register_file_write_data_select),
.MEM_register_write_enable(MEM_register_write_enable),
.MEM_csr_write_enable(MEM_csr_write_enable),
.MEM_opcode(MEM_opcode),
.MEM_funct3(MEM_funct3),
.MEM_rs1(MEM_rs1),
.MEM_rd(MEM_rd),
.MEM_raw_imm(MEM_raw_imm),
.MEM_read_data2(MEM_read_data2),
.MEM_imm(MEM_imm),
.MEM_csr_read_data(MEM_csr_read_data),
.MEM_alu_result(MEM_alu_result)
);

MEM_WB_Register #(XLEN(XLEN)) mem_wb_register (
    .clk(clk),
        .reset(reset),
    .MEM_WB_stall(MEM_WB_stall),
    .flush(MEM_WB_flush),

// Signals from EX_MEM_Register
.MEM_pc(MEM_pc),
.MEM_pc_plus_4(MEM_pc_plus_4),
.MEM_instruction(MEM_instruction),

.MEM_register_file_write_data_select(MEM_register_file_write_data_select),
.MEM_imm(MEM_imm),
.MEM_csr_read_data(MEM_csr_read_data),
.MEM_alu_result(MEM_alu_result),
.MEM_register_write_enable(MEM_register_write_enable),
.MEM_csr_write_enable(MEM_csr_write_enable),
.MEM_rs1(MEM_rs1),
.MEM_rd(MEM_rd),
.MEM_raw_imm(MEM_raw_imm),
.MEM_opcode(MEM_opcode),

// Signal from MEM Phase
.MEM_byte_enable_logic_register_file_write_data(byte_enable_logic_register_file_write_data),

// Signals to WB Phase
.WB_pc(WB_pc),
.WB_pc_plus_4(WB_pc_plus_4),
.WB_instruction(WB_instruction),
.WB_register_file_write_data_select(WB_register_file_write_data_select),

```

```

.WB_imm(WB_imm),
.WB_csr_read_data(WB_csr_read_data),
.WB_alu_result(WB_alu_result),
.WB_register_write_enable(WB_register_write_enable),
.WB_csr_write_enable(WB_csr_write_enable),
.WB_rs1(WB_rs1),
.WB_rd(WB_rd),
.WB_raw_imm(WB_raw_imm),
.WB_opcode(WB_opcode),
.WB_byte_enable_logic_register_file_write_data(WB_byte_enable_logic_register_file_write_data)
);

```

```

HazardUnit hazard_unit (
.clk(clk),
.reset(reset),
.trap_done(trap_done),
.standby_mode(standby_mode),
.trap_status(trap_status),
.misaligned_instruction_flush(misaligned_instruction_flush),
.misaligned_memory_flush(misaligned_memory_flush),
.pth_done_flush(pth_done_flush),
.csr_ready(csr_ready),
.ID_rs1(rs1),
.ID_rs2(rs2),
.ID_raw_imm(raw_imm[11:0]),
.EX_csr_write_enable(EX_csr_write_enable),
.MEM_rd(MEM_rd),
.MEM_register_write_enable(MEM_register_write_enable),
.MEM_csr_write_enable(MEM_csr_write_enable),
.MEM_csr_write_address(MEM_raw_imm[11:0]),
.WB_rd(WB_rd),
.WB_register_write_enable(WB_register_write_enable),
.WB_csr_write_enable(WB_csr_write_enable),
.WB_csr_write_address(WB_raw_imm[11:0]),
.EX_rs1(EX_rs1),
.EX_rs2(EX_rs2),
.EX_rd(EX_rd),
.EX_opcode(EX_opcode),
.EX_imm(EX_raw_imm[11:0]),
.branch_prediction_miss(branch_prediction_miss),
.EX_jump(EX_jump),

.hazard_mem(hazard_mem),
.hazard_wb(hazard_wb),
.csr_hazard_mem(csr_hazard_mem),
.csr_hazard_wb(csr_hazard_wb),
//.csr_reg_hazard(csr_reg_hazard),

.IF_ID_flush(IF_ID_flush),
.ID_EX_flush(ID_EX_flush),
.EX_MEM_flush(EX_MEM_flush),
.MEM_WB_flush(MEM_WB_flush),
.IF_ID_stall(IF_ID_stall),
.ID_EX_stall(ID_EX_stall),

```

```

.EX_MEM_stall(EX_MEM_stall),
.MEM_WB_stall(MEM_WB_stall)
);

```

```

ForwardUnit forward_unit (
.hazard_mem(hazard_mem),
.hazard_wb(hazard_wb),
.MEM_imm(MEM_imm),
.MEM_alu_result(MEM_alu_result),
.MEM_csr_read_data(MEM_csr_read_data),
.byte_enable_logic_register_file_write_data(byte_enable_logic_register_file_write_data),
.MEM_pc_plus_4(MEM_pc_plus_4),
.MEM_opcode(MEM_opcode),
.WB_opcode(WB_opcode),
.WB_imm(WB_imm),
.WB_alu_result(WB_alu_result),
.WB_csr_read_data(WB_csr_read_data),
.WB_byte_enable_logic_register_file_write_data(WB_byte_enable_logic_register_file_write_data),
.WB_pc_plus_4(WB_pc_plus_4),
.alu_forward_source_data_a(alu_forward_source_data_a),
.alu_forward_source_data_b(alu_forward_source_data_b),
.alu_forward_source_select_a(alu_forward_source_select_a),
.alu_forward_source_select_b(alu_forward_source_select_b),

.csr_hazard_mem(csr_hazard_mem),
.csr_hazard_wb(csr_hazard_wb),
.MEM_csr_write_data(MEM_alu_result),
.WB_csr_write_data(WB_alu_result),
.csr_read_data(csr_read_out),

.csr_forward_data(csr_forward_data)
);

```

```

BranchPredictor #(XLEN(XLEN)) branch_predictor(
.clk(clk),
.reset(reset),
.IF_opcode(IF_opcode),
.IF_pc (pc),
.IF_imm (IF_imm),
.EX_branch(EX_branch),
.EX_branch_taken (branch_taken),

.branch_estimation (branch_estimation),
.branch_target (branch_target)
);

```

```

always @(posedge clk or posedge reset) begin
if (reset) begin
retired_alu_result <= {XLEN{1'b0}};
retired_csr_read_data <= {XLEN{1'b0}};
end else begin
retired_alu_result <= WB_alu_result;
retired_csr_read_data <= WB_csr_read_data;
end
end

```

```

end

always @(*) begin
    if (EX_alu_src_A_select == `ALU_SRC_A_RD1) begin
        alu_normal_source_a = EX_read_data1;
    end
    else if (EX_alu_src_A_select == `ALU_SRC_A_PC) begin
        alu_normal_source_a = EX_pc;
    end
    else if (EX_alu_src_A_select == `ALU_SRC_A_RS1) begin
        alu_normal_source_a = {27'b0, EX_rs1};
    end
    else begin
        alu_normal_source_a = 32'b0;
    end

    if (EX_alu_src_B_select == `ALU_SRC_B_RD2) begin
        alu_normal_source_b = EX_read_data2;
    end
    else if (EX_alu_src_B_select == `ALU_SRC_B_IMM) begin
        alu_normal_source_b = EX_imm;
    end
    else if (EX_alu_src_B_select == `ALU_SRC_B_SHAMT) begin
        alu_normal_source_b = {27'b0, EX_imm[4:0]};
    end
    else if (EX_alu_src_B_select == `ALU_SRC_B_CSR) begin
        alu_normal_source_b = csr_forward_data;
    end
    else begin
        alu_normal_source_b = 32'b0;
    end

    if (!standby_mode && trapped) begin
        csr_write_data = csr_trap_write_data;
        csr_write_address = csr_trap_address;
        csr_read_address = csr_trap_address;
    end
    else begin
        csr_write_data = WB_alu_result;
        csr_write_address = WB_raw_imm[11:0];
        csr_read_address = raw_imm[11:0];
    end

    if (debug_mode) instruction = dbg_instruction;
    else instruction = im_instruction;

    case (WB_register_file_write_data_select)
        `RF_WD_LOAD: begin
            register_file_write_data = WB_byte_enable_logic_register_file_write_data;
        end
        `RF_WD_ALU: begin
            register_file_write_data = WB_alu_result;
        end
        `RF_WD_LUI: begin

```

```

        register_file_write_data = WB_imm;
    end
    `RF_WD_JUMP: begin
        register_file_write_data = WB_pc_plus_4;
    end
    `RF_WD_CSR: begin
        /*if (csr_reg_hazard) begin
            register_file_write_data = retired_alu_result;
        end else */begin
            register_file_write_data = WB_csr_read_data;
        end
    end
    default: begin
        register_file_write_data = 32'b0;
    end
endcase

case (alu_forward_source_select_a)
    2'b10: src_A = alu_forward_source_data_a;
    2'b11: src_A = alu_forward_source_data_a;
    default: src_A = alu_normal_source_a;
endcase

case (alu_forward_source_select_b)
    2'b10: src_B = alu_forward_source_data_b;
    2'b11: src_B = alu_forward_source_data_b;
    default: src_B = alu_normal_source_b;
endcase
end

endmodule

```

Program_Counter.v

```

module ProgramCounter (
    input clk,
    input reset,
    input [31:0] next_pc, // Next pc value
    output reg [31:0] pc // Current pc value
);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        pc <= 32'b0; // Reset to 0
    end
    else begin
        pc <= next_pc; // Update pc value
    end
end

endmodule

```

PC_Plus_4.v

```

module PCPlus4 (
    input [31:0] pc, // Current pc value

```

```

        output reg [31:0] pc_plus_4 // pc+4 value
    );

    always @(*) begin
        pc_plus_4 = pc + 4;
    end

endmodule

```

PC_Controller.v

```

module PCController (
    input jump, // signal indicating if PC should jump
    input branch_estimation, // signal indicating if PC should take the branch
    input branch_prediction_miss,
    input trapped, // signal indicating if trap has occurred
    input [31:0] pc, // current pc value
    input [31:0] jump_target, // target address for jump
    input [31:0] branch_target, // target address for branch from branch predictor
    input [31:0] branch_target_actual, // actual branch target address when mispredicted
    input [31:0] trap_target, // target address for trap
    input pc_stall, // signal indicating if pc update should be paused

    output reg [31:0] next_pc // next pc value
);

always @(*) begin
    if (!pc_stall) begin
        if (trapped) begin
            next_pc = trap_target;
        end
        else if (trapped == 1'b0 && jump) begin
            next_pc = jump_target;
        end
        else if (trapped == 1'b0 && branch_prediction_miss) begin
            next_pc = branch_target_actual;
        end
        else if (trapped == 1'b0 && branch_estimation) begin
            next_pc = branch_target;
        end
        else begin
            next_pc = pc + 4;
        end
    end
    else begin
        next_pc = pc;
    end
end

endmodule

```

PC_Aligner.v


```

module PCAligner (
    input [31:0] raw_next_pc,
    output reg [31:0] next_pc
);

    always @(*) begin
        next_pc = {raw_next_pc[31:2], 2'b0};
    end

endmodule

```

IF_ID.v

```

module IF_ID_Register #(
    parameter XLEN = 32
)(
    // pipeline register control signals
    input wire clk,
    input wire reset,
    input wire flush,
    input wire IF_ID_stall,

    // signals from IF phase
    input wire [XLEN-1:0] IF_pc,
    input wire [XLEN-1:0] IF_pc_plus_4,
    input wire [31:0] IF_instruction,
    input wire IF_branch_estimation,

    // signals to ID/EX register
    output reg [XLEN-1:0] ID_pc,
    output reg [XLEN-1:0] ID_pc_plus_4,
    output reg [31:0] ID_instruction,
    output reg ID_branch_estimation
);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        ID_pc <= {XLEN{1'b0}};
        ID_pc_plus_4 <= {XLEN{1'b0}};
        ID_instruction <= 32'h0000_0013;
        ID_branch_estimation <= 1'b0;
    end
    else begin
        if (flush) begin
            ID_pc <= {XLEN{1'b0}};
            ID_pc_plus_4 <= {XLEN{1'b0}};
            ID_instruction <= 32'h0000_0013; // ADDI x0, x0, 0 = RISC-V NOP, HINT
            ID_branch_estimation <= 1'b0;
        end else if (!IF_ID_stall) begin
            ID_pc <= IF_pc;
            ID_pc_plus_4 <= IF_pc_plus_4;
            ID_instruction <= IF_instruction;
            ID_branch_estimation <= IF_branch_estimation;
        end
    end
end

```

```

    end
end

endmodule

```

Byte_Enable_Logic.v

```

`include "modules/headers/load.vh"
`include "modules/headers/store.vh"

module ByteEnableLogic (
    input memory_read,                // signal indicating that register file
    should_read_from_data_memory,
    input memory_write,              // signal indicating that register file
    should_write_to_data_memory,
    input [2:0] funct3,              // funct3
    input [31:0] register_file_read_data, // data read from register file
    input [31:0] data_memory_read_data, // data read from data memory
    input [1:0] address,              // address for checking alignment

    output reg [31:0] register_file_write_data, // data to write at register file
    output reg [31:0] data_memory_write_data, // data to write at data memory
    output reg [3:0] write_mask        // bitmask for writing data
);

reg [7:0] byte_sel;
reg [15:0] half_sel;

always @(*) begin
    if (memory_read) begin
        data_memory_write_data = 32'b0;
        write_mask = 4'b0;

        case (funct3)
            `LOAD_LB, `LOAD_LBU : begin
                case (address[1:0])
                    2'b00: byte_sel = data_memory_read_data[ 7: 0];
                    2'b01: byte_sel = data_memory_read_data[15: 8];
                    2'b10: byte_sel = data_memory_read_data[23:16];
                    2'b11: byte_sel = data_memory_read_data[31:24];
                endcase

                if (funct3 == `LOAD_LBU) begin
                    register_file_write_data = {24'b0, byte_sel}; // zero-extend
                end else begin
                    register_file_write_data = {{24{byte_sel[7]}}, byte_sel}; // sign-
                end
            end

            `LOAD_LH, `LOAD_LHU : begin
                case (address[1])
                    1'b0 : half_sel = data_memory_read_data[15:0];
                    1'b1 : half_sel = data_memory_read_data[31:16];
                endcase
            end
        endcase
    end
end

```

```

        endcase

        if (funct3 == `LOAD_LHU) begin
            register_file_write_data = {16'b0, half_sel};          // zero-extend
        end else begin
            register_file_write_data = {{16{half_sel[15]}}, half_sel}; // sign-
        end

    end

end

`LOAD_LW : begin
    register_file_write_data = data_memory_read_data;
end

default: begin
    register_file_write_data = 32'b0;
end

endcase

end

else if (memory_write) begin
    register_file_write_data = 32'b0;

    case (funct3)
        `STORE_SB: begin
            data_memory_write_data = {4{register_file_read_data[7:0]}};

            case (address[1:0])
                2'b00: begin
                    write_mask = 4'b0001;
                end
                2'b01: begin
                    write_mask = 4'b0010;
                end
                2'b10: begin
                    write_mask = 4'b0100;
                end
                2'b11: begin
                    write_mask = 4'b1000;
                end
            endcase
        end

        `STORE_SH: begin
            data_memory_write_data = {2{register_file_read_data[15:0]}};

            case (address[1:0])
                2'b00: begin
                    write_mask = 4'b0011;
                end
                2'b10: begin
                    write_mask = 4'b1100;
                end
                default: begin
                    write_mask = 4'b0;
                end
            endcase
        end
    endcase
end

```

```

                                endcase
                            end
                            `STORE_SW: begin
                                data_memory_write_data = register_file_read_data;

                                if (address[1:0] == 2'b00) begin
                                    write_mask = 4'b1111;
                                end
                                else begin
                                    write_mask = 4'b0;
                                end
                            end
                            default: begin
                                data_memory_write_data = 32'b0;
                                write_mask = 4'b0;
                            end
                        endcase
                    end
                    else begin
                        register_file_write_data = 32'b0;
                        data_memory_write_data = 32'b0;
                        write_mask = 4'b0;
                        byte_sel = 8'b0;
                        half_sel = 16'b0;
                    end
                end
            end
        end
    endmodule

```

Data_Memory.v

```

module DataMemory (
    input clk,                // clock signal
    input write_enable,       // enabling signal for writing Data Memory
    input [31:0] address,     // Take address of memory to read/write value
    input [31:0] write_data,  // data to write to Data Memory
    input [3:0] write_mask,   // bitmask for writing data (Should receive 4'b1111 in RV32I47F and 50F top module)

    input [31:0] rom_read_data,
    output [31:0] rom_address,

    output reg [31:0] read_data // data read from Data Memory
);

reg [31:0] memory [0:1023]; // 1024 words (4KB), FPGA is 32KB = 8192 (0:8191)
reg [31:0] new_word;

// 32 bit extended mask from 4 bit write_mask
wire [31:0] extended_mask = {{8{write_mask[3]}}, {8{write_mask[2]}}, {8{write_mask[1]}}, {8{write_mask[0]}}};
wire ram_access = (address[31:16] == 16'h1000);
wire rom_access = (address[31:16] == 16'h0000);
wire [12:0] ram_address = address[14:2];
assign rom_address = address;

integer i;

```

```

initial begin
    for (i=0; i<1024; i=i+1) begin
        memory[i] = 32'b0;
    end // $readmemh("./data_init.mem", memory, 13'h1424);
    new_word = 32'b0;
end

always @(*) begin
    if (ram_access) begin
        read_data = memory[ram_address];
    end else if (rom_access) begin
        read_data = rom_read_data;
    end else begin
        read_data = 32'b0; // for wrong address
    end
end

end

always @(posedge clk) begin
    if (write_enable && ram_access) begin
        new_word = ((memory[ram_address] & ~extended_mask) | (write_data & extended_mask));
        memory[ram_address] <= new_word;
    end
end

endmodule

```

Instruction_Memory.v

```

`include "modules/headers/branch.vh"
`include "modules/headers/itype.vh"
`include "modules/headers/load.vh"
`include "modules/headers/rtype.vh"
`include "modules/headers/store.vh"
`include "modules/headers/opcode.vh"
`include "modules/headers/csr.vh"

module InstructionMemory (
    input [31:0] pc,
    output reg [31:0] instruction,

    input [31:0] rom_address,
    output reg [31:0] rom_read_data
);

    reg [31:0] data [0:2047];    // in FPGA, 16384.
    integer i;
    initial begin
        // $readmemh("./dhrystone.mem", data);
        for (i=0; i<2048; i=i+1) begin
            data[i] = {12'h2BC, 5'd0, `ITYPE_ADDI, 5'd0, `OPCODE_ITYPE};
        end
        // _____
        // I-타입 ALU 명령어 (9개)
    end
endmodule

```

```

// {imm[11:0], rs1, funct3, rd, OPCODE_ITYPE}
data[0] = {12'h2BC, 5'd0, 'ITYPE_ADDI, 5'd1, 'OPCODE_ITYPE}; // ADDI:
x1 = x0 + 2BC = 000002BC
data[1] = {12'd24, 5'd1, 'ITYPE_SLLI, 5'd2, 'OPCODE_ITYPE}; // SLLI:
x2 = x1 << 24 = BC000000
data[2] = {12'd0, 5'd2, 'ITYPE_SLTI, 5'd3, 'OPCODE_ITYPE}; // SLTI:
x3 = (x2(-1140850688d) < 0) ? 1 : 0 = 00000001
data[3] = {12'd0, 5'd2, 'ITYPE_SLTIU, 5'd4, 'OPCODE_ITYPE}; // SLTIU:
x4 = (x2(3154116608d) < 0) ? 1 : 0 = 00000000
data[4] = {12'h653, 5'd1, 'ITYPE_XORI, 5'd5, 'OPCODE_ITYPE}; // XORI: x5 = x1
XOR 653 = 000004EF
data[5] = {7'b0000000, 5'd4, 5'd2, 'ITYPE_SRXI, 5'd6, 'OPCODE_ITYPE}; // SRLI: x6 = x2 >> 4 =
0BC00000
data[6] = {7'b0100000, 5'd4, 5'd2, 'ITYPE_SRXI, 5'd7, 'OPCODE_ITYPE}; // SRAI: x7 = x2 >>> 4 =
FBC00000
data[7] = {12'h0BC, 5'd2, 'ITYPE_ORI, 5'd8, 'OPCODE_ITYPE}; // ORI:
x8 = x2 OR BC = BC0000BC
data[8] = {12'h0EC, 5'd5, 'ITYPE_ANDI, 5'd9, 'OPCODE_ITYPE}; // ANDI:
x9 = x5 AND 0EC = 000000EC

```

// R-타입 명령어 (10개)

```

// {funct7, rs2, rs1, funct3, rd, OPCODE_RTYPE}
data[9] = {7'b0000000, 5'd9, 5'd1, 'RTYPE_ADDSUB, 5'd10, 'OPCODE_RTYPE}; // ADD: x10 = x1 +
x9 = 000003A8
data[10] = {7'b0100000, 5'd5, 5'd6, 'RTYPE_ADDSUB, 5'd11, 'OPCODE_RTYPE}; // SUB: x11 = x6 -
x5 = 0BBFFB11
data[11] = {7'b0000000, 5'd3, 5'd7, 'RTYPE_SLL, 5'd12, 'OPCODE_RTYPE}; // SLL:
x12 = x7 << x3 = F7800000
data[12] = {7'b0000000, 5'd2, 5'd1, 'RTYPE_SLT, 5'd13, 'OPCODE_RTYPE}; // SLT:
x13 = (x1 < x2) ? 1 : 0 = 00000000
data[13] = {7'b0000000, 5'd2, 5'd1, 'RTYPE_SLTU, 5'd14, 'OPCODE_RTYPE}; // SLTU:
x14 = (x1 < x2 unsigned) ? 1 : 0 = 00000001
data[14] = {7'b0000000, 5'd8, 5'd12, 'RTYPE_XOR, 5'd15, 'OPCODE_RTYPE}; // XOR:
x15 = x12 XOR x8 = 4B8000BC
data[15] = {7'b0000000, 5'd3, 5'd12, 'RTYPE_SR, 5'd16, 'OPCODE_RTYPE}; // SRL:
x16 = x12 >> x3 = 7BC00000
data[16] = {7'b0100000, 5'd3, 5'd12, 'RTYPE_SR, 5'd17, 'OPCODE_RTYPE}; // SRA:
x17 = x12 >>> x3 = FBC00000
data[17] = {7'b0000000, 5'd7, 5'd11, 'RTYPE_OR, 5'd18, 'OPCODE_RTYPE}; // OR:
x18 = x11 OR x7 = FBFFFB11
data[18] = {7'b0000000, 5'd11, 5'd7, 'RTYPE_AND, 5'd19, 'OPCODE_RTYPE}; // AND:
x19 = x7 AND x11 = 0B800000

```

// S-타입 명령어 (스토어) (3개)

```

// {imm[11:5], rs2, rs1, funct3, imm[4:0], OPCODE_STORE}
data[19] = {7'd0, 5'd11, 5'd1, 'STORE_SW, 5'd4, 'OPCODE_STORE};
// SW: mem[x1+4 = 2C0] = (x11 = 0BBFFB11) -> 0BBFFB11
data[20] = {7'd0, 5'd10, 5'd1, 'STORE_SH, 5'd7, 'OPCODE_STORE};
// SH: mem[x1+7 = 2C3 (write nothing)] = (x10[15:0] = 03A8) -> 0BBFFB11 // Misaligned Memory exception. NOPs
and goes to Trap Handler.
data[21] = {7'd0, 5'd15, 5'd1, 'STORE_SB, 5'd4, 'OPCODE_STORE};
// SB: mem[x1+4 = 2C0] = (x15[7:0] = BC) -> 0BBFFBBC

```

```

// -----
// I-타입 로드 명령어 (5개) - RAM 접근을 위해 x1 주소 변경
// {imm[11:0], rs1, funct3, rd, OPCODE_LOAD}

// x1을 1000_02BC로 변경 (RAM 영역 접근용)
data[22] = {20'h10000, 5'd31, `OPCODE_LUI};
// LUI: x31 = 0x10000000
data[23] = {7'b0000000, 5'd31, 5'd1, `RTYPE_OR, 5'd1, `OPCODE_RTYPE}; // OR: x1 = x1 |
x31 = 0x100002BC

data[24] = {12'd5, 5'd1, `LOAD_LW, 5'd20, `OPCODE_LOAD};
// LW: x20 = mem[x1+5 = 100002C1] = misaligned exception
data[25] = {12'd4, 5'd1, `LOAD_LH, 5'd21, `OPCODE_LOAD};
// LH: x21 = mem[x1+4 = 100002C0][15:0]
data[26] = {12'd4, 5'd1, `LOAD_LB, 5'd22, `OPCODE_LOAD};
// LB: x22 = mem[x1+4 = 100002C0][7:0]
data[27] = {12'd4, 5'd1, `LOAD_LHU, 5'd23, `OPCODE_LOAD};
// LHU: x23 = mem[x1+4 = 100002C0][15:0]
data[28] = {12'd4, 5'd1, `LOAD_LBU, 5'd24, `OPCODE_LOAD};
// LBU: x24 = mem[x1+4 = 100002C0][7:0]

// x1을 다시 0000_02BC로 복원
data[29] = {12'h2BC, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};
// ADDI: x1 = 0x000002BC

// -----
// U-타입 명령어 (2개)
// {imm[31:12], rd, OPCODE_LUI/OPCODE_AUIPC}
data[30] = {20'd1, 5'd25, `OPCODE_LUI};
// LUI: x25 = 00001000
data[31] = {20'd1, 5'd26, `OPCODE_AUIPC};
// AUIPC: x26 = 0000007C + 00001000 = 0000107C

// -----
// J-타입 명령어 (1개)
// {imm[20:1|11|19:12], rd, OPCODE_JAL}
data[32] = {20'b0_0000001111_0_00000000, 5'd27, `OPCODE_JAL};
// JAL: PC(0x80) + 0x1E = 0x9E (misaligned), x27 = PC + 4 = 0x84
// But since this instruction occurs exception, It's handled as NOP.

// -----
// B-타입 명령어 (분기) (6개)
// {imm[12], imm[10:5], rs2, rs1, funct3, imm[4:1], imm[11], OPCODE_BRANCH}
data[33] = {1'b0, 6'd0, 5'd2, 5'd1, `BRANCH_BEQ, 4'b0100, 1'b0, `OPCODE_BRANCH}; // BEQ:
if(x1 == x2) branch offset = 8 Not Taken
data[34] = {1'b0, 6'd0, 5'd13, 5'd0, `BRANCH_BNE, 4'b0100, 1'b0, `OPCODE_BRANCH}; // BNE:
if(x0 != x13) branch offset = 8 Not Taken
data[35] = {1'b0, 6'd0, 5'd2, 5'd1, `BRANCH_BLT, 4'b0100, 1'b0, `OPCODE_BRANCH}; // BLT:
if(x1 < x2) branch offset = 8 Not Taken (x2 = signed negative)
data[36] = {1'b0, 6'd0, 5'd1, 5'd2, `BRANCH_BGE, 4'b0100, 1'b0, `OPCODE_BRANCH}; // BGE:
if(x2 >= x1) branch offset = 8 Not Taken (x2 = signed negative)

```

```

        data[37] = {1'b0, 6'd0, 5'd1, 5'd2, `BRANCH_BLTU, 4'b0100, 1'b0, `OPCODE_BRANCH}; // BLTU:
if(x2 < x1 unsigned) branch offset = 8          Not Taken
        data[38] = {1'b0, 6'd0, 5'd1, 5'd2, `BRANCH_BGEU, 4'b0100, 1'b0, `OPCODE_BRANCH}; // BGEU:
if(x2 >= x1 unsigned) branch offset = 8 Taken -> data[40]

```

```

// -----
// I-타입 점프 (JALR) 명령어 (1개)
// {imm[11:0], rs1, funct3, rd, OPCODE_JALR}
data[39] = {12'd0, 5'd27, 3'b000, 5'd28, `OPCODE_JALR};
// JALR: x28 = PC + 4 = 0xA0; PC = x27 + 0 = 0x84

```

```

// -----
// I-타입 Zicsr 확장 명령어 (6개)      [F11] == mvendorid, [341] = mepc, [342] =
mcause, [305] = mtvec
// {imm[11:0], rs1(uimm), funct3, rd, OPCODE_ENVIRONMENT}
data[40] = {12'hF11, 5'd28, `CSR_CSRRW, 5'd20, `OPCODE_ENVIRONMENT}; //
CSRWW : x28 = 0000_0000; x20 = 5256_4B43 <= CSR[F11] = 5256_4B43 // R[x20] = 5256_4B43.
data[41] = {12'h341, 5'd1, `CSR_CSRRS, 5'd21, `OPCODE_ENVIRONMENT};
// CSRRS: x1 = 0000_02BC; CSR[341] = 0000_0074. // R[x21] =
0000_0074, CSR[341] = 0000_02fc
data[42] = {12'h341, 5'd20, `CSR_CSRRC, 5'd21, `OPCODE_ENVIRONMENT}; //
CSRRC: x21 = 0000_0074, x20 = 5256_4B43, CSR[341] = 0000_02fc. // R[x21] = 0000_02fc, CSR[341] = 0000_00BC
data[43] = {12'h342, 5'd3, `CSR_CSRRWI, 5'd22, `OPCODE_ENVIRONMENT}; //
CSRWI: x22 = FFFF_FFBC, CSR[342] = 0000_0000; // R[x22] = 0000_0000,
CSR[342] = 0000_0003
data[44] = {12'h305, 5'd7, `CSR_CSRRSI, 5'd22, `OPCODE_ENVIRONMENT}; //
CSRSI: x22 = 0000_0000, CSR[305] = 0000_1000; // R[x22] = 0000_1000,
CSR[305] = 0000_1007
data[45] = {12'h305, 5'b11111, `CSR_CSRRCI, 5'd23, `OPCODE_ENVIRONMENT}; // CSRRCI: uimm
= 11111, CSR[305] = 0000_1007; // R[x23] = 0000_1007, CSR[305] =
0000_1000

```

```

// -----
// I-타입 HINT 명령어 (CSR 동작 확인)
// {imm[11:0], rs1, funct3, rd, OPCODE_ITYPE}
data[46] = {12'h2BC, 5'd0, `ITYPE_ADDI, 5'd0, `OPCODE_ITYPE};
// ADDI: x0 = x0 + 2BC = 0000_0000

```

```

// -----
// ECALL 명령어, Misaligned Instruction address exception 발생 JALR 명령어,
Misaligned Memory Address access exception 발생 SH 명령어

```

```

data[47] = {12'd0, 5'd0, 3'd0, 5'd0, `OPCODE_ENVIRONMENT};
// ECALL: PC = CSR[mtvec] = 0000_1000 = data[1024]
data[48] = {12'd1, 5'd27, 3'b000, 5'd28, `OPCODE_JALR};
// JALR: x28 = PC + 4 = 0xC4; PC = x27 + 1 = 0x85 -> misaligned
data[49] = {7'b0, 5'd5, 5'd1, `STORE_SH, 5'd1, `OPCODE_STORE};
// SH: mem[x1+1 = 2BD, misaligned] = (x5[15:0] = 04EF) misaligned exception..

```

```

// -----
// Debug Interface 명령어 수행을 위한 전초 작업. 기존 x22 값 FFFF_FFBC 값을 더하는 ADD
명령어를 DI에서 수행할 예정
data[50] = {20'd0, 5'd22, `OPCODE_LUI};
// LUI: x22 = 0000_0000

```



```

        data[51] = {12'hFBC, 5'd22, `ITYPE_ADDI, 5'd22, `OPCODE_ITYPE};
// ADDI x22 = x22 - 0x44 = FFFF_FFBC
        data[52] = {20'hABADC, 5'd23, `OPCODE_LUI};
        // LUI: x23 = ABAD_C000
        data[53] = {12'hB02, 5'd23, `ITYPE_ADDI, 5'd23, `OPCODE_ITYPE};
// ADDI: x23 = x23 + -4FE = ABAD_BB02

// -----
// printf 수행을 위한 MMIO Interface 테스트벤치 명령어. SB to 0x10010000, store
"ABADBEBE" = UART로 출력
        data[54] = {1'b0, 10'b0000000110, 1'b0, 8'b0, 5'd0, `OPCODE_JAL};
// JAL x0, +12: data[57]로 분기

        data[55] = {12'd1, 5'd0, 3'd0, 5'd0, `OPCODE_ENVIRONMENT};
// EBREAK:

                                                                 // LADD: x22 = x22 + x23. FFFF_FFBC(x22) +
ABAD_BB02(x23) = ABAD_BABE(x22)

        // HINT; NOP for 'x' signal after EBREAK in pipeline
        data[56] = {12'h2BC, 5'd0, `ITYPE_ADDI, 5'd0, `OPCODE_ITYPE};
// ADDI: x0 = x0 + 2BC = 0000_0000

// -----
// UART MMIO 테스트: 0x10010000에 "ABADBEBE" 출력 (Polling 추가)
        data[57] = {20'h10010, 5'd29, `OPCODE_LUI};
        // LUI: x29 = 0x10010000 (UART TX Data 주소)
        data[58] = {12'h004, 5'd29, `ITYPE_ADDI, 5'd28, `OPCODE_ITYPE};
// ADDI: x28 = x29 + 4 = 0x10010004 (UART Status 주소)

// 'A' 전송
        data[59] = {12'h041, 5'd0, `ITYPE_ADDI, 5'd31, `OPCODE_ITYPE};
// ADDI: x31 = 'A' (0x41)
        data[60] = {12'd0, 5'd28, `LOAD_LW, 5'd30, `OPCODE_LOAD};
// LW x30, 0(x28): Status 레지스터 읽기
        data[61] = {12'd1, 5'd30, `ITYPE_ANDI, 5'd30, `OPCODE_ITYPE};
// ANDI x30, x30, 1: busy bit 마스크
        data[62] = {1'b1, 6'b111111, 5'd0, 5'd30, `BRANCH_BNE, 4'b1100, 1'b1,
`OPCODE_BRANCH}; // BNE x30, x0, -8: busy이면 data[60]로 재시도
        data[63] = {7'd0, 5'd31, 5'd29, `STORE_SB, 5'd0, `OPCODE_STORE};
// SB: mem[x29+0] = 'A'

// 'B' 전송
        data[64] = {12'h042, 5'd0, `ITYPE_ADDI, 5'd31, `OPCODE_ITYPE};
// ADDI: x31 = 'B' (0x42)
        data[65] = {12'd0, 5'd28, `LOAD_LW, 5'd30, `OPCODE_LOAD};
// LW x30, 0(x28): Status 레지스터 읽기
        data[66] = {12'd1, 5'd30, `ITYPE_ANDI, 5'd30, `OPCODE_ITYPE};
// ANDI x30, x30, 1: busy bit 마스크
        data[67] = {1'b1, 6'b111111, 5'd0, 5'd30, `BRANCH_BNE, 4'b1100, 1'b1,
`OPCODE_BRANCH}; // BNE x30, x0, -8: busy이면 data[65]로 재시도

```

```

        data[68] = {7'd0, 5'd31, 5'd29, `STORE_SB, 5'd0, `OPCODE_STORE};
// SB: mem[x29+0] = 'B'

        // 'A' 전송
        data[69] = {12'h041, 5'd0, `ITYPE_ADDI, 5'd31, `OPCODE_ITYPE};
// ADDI: x31 = 'A' (0x41)
        data[70] = {12'd0, 5'd28, `LOAD_LW, 5'd30, `OPCODE_LOAD};
// LW x30, 0(x28): Status 레지스터 읽기
        data[71] = {12'd1, 5'd30, `ITYPE_ANDI, 5'd30, `OPCODE_ITYPE};
// ANDI x30, x30, 1: busy bit 마스크
        data[72] = {1'b1, 6'b111111, 5'd0, 5'd30, `BRANCH_BNE, 4'b1100, 1'b1,
`OPCODE_BRANCH}; // BNE x30, x0, -8: busy이면 data[70]로 재시도
        data[73] = {7'd0, 5'd31, 5'd29, `STORE_SB, 5'd0, `OPCODE_STORE};
// SB: mem[x29+0] = 'A'

        // 'D' 전송
        data[74] = {12'h044, 5'd0, `ITYPE_ADDI, 5'd31, `OPCODE_ITYPE};
// ADDI: x31 = 'D' (0x44)
        data[75] = {12'd0, 5'd28, `LOAD_LW, 5'd30, `OPCODE_LOAD};
// LW x30, 0(x28): Status 레지스터 읽기
        data[76] = {12'd1, 5'd30, `ITYPE_ANDI, 5'd30, `OPCODE_ITYPE};
// ANDI x30, x30, 1: busy bit 마스크
        data[77] = {1'b1, 6'b111111, 5'd0, 5'd30, `BRANCH_BNE, 4'b1100, 1'b1,
`OPCODE_BRANCH}; // BNE x30, x0, -8: busy이면 data[75]로 재시도
        data[78] = {7'd0, 5'd31, 5'd29, `STORE_SB, 5'd0, `OPCODE_STORE};
// SB: mem[x29+0] = 'D'

        // 'B' 전송
        data[79] = {12'h042, 5'd0, `ITYPE_ADDI, 5'd31, `OPCODE_ITYPE};
// ADDI: x31 = 'B' (0x42)
        data[80] = {12'd0, 5'd28, `LOAD_LW, 5'd30, `OPCODE_LOAD};
// LW x30, 0(x28): Status 레지스터 읽기
        data[81] = {12'd1, 5'd30, `ITYPE_ANDI, 5'd30, `OPCODE_ITYPE};
// ANDI x30, x30, 1: busy bit 마스크
        data[82] = {1'b1, 6'b111111, 5'd0, 5'd30, `BRANCH_BNE, 4'b1100, 1'b1,
`OPCODE_BRANCH}; // BNE x30, x0, -8: busy이면 data[80]로 재시도
        data[83] = {7'd0, 5'd31, 5'd29, `STORE_SB, 5'd0, `OPCODE_STORE};
// SB: mem[x29+0] = 'B'

        // 'E' 전송
        data[84] = {12'h045, 5'd0, `ITYPE_ADDI, 5'd31, `OPCODE_ITYPE};
// ADDI: x31 = 'E' (0x45)
        data[85] = {12'd0, 5'd28, `LOAD_LW, 5'd30, `OPCODE_LOAD};
// LW x30, 0(x28): Status 레지스터 읽기
        data[86] = {12'd1, 5'd30, `ITYPE_ANDI, 5'd30, `OPCODE_ITYPE};
// ANDI x30, x30, 1: busy bit 마스크
        data[87] = {1'b1, 6'b111111, 5'd0, 5'd30, `BRANCH_BNE, 4'b1100, 1'b1,
`OPCODE_BRANCH}; // BNE x30, x0, -8: busy이면 data[85]로 재시도
        data[88] = {7'd0, 5'd31, 5'd29, `STORE_SB, 5'd0, `OPCODE_STORE};
// SB: mem[x29+0] = 'E'

```

```

// 'B' 전송
data[89] = {12'h042, 5'd0, `ITYPE_ADDI, 5'd31, `OPCODE_ITYPE};
// ADDI: x31 = 'B' (0x42)
data[90] = {12'd0, 5'd28, `LOAD_LW, 5'd30, `OPCODE_LOAD};
// LW x30, 0(x28): Status 레지스터 읽기
data[91] = {12'd1, 5'd30, `ITYPE_ANDI, 5'd30, `OPCODE_ITYPE};
// ANDI x30, x30, 1: busy bit 마스크
data[92] = {1'b1, 6'b111111, 5'd0, 5'd30, `BRANCH_BNE, 4'b1100, 1'b1,
`OPCODE_BRANCH}; // BNE x30, x0, -8: busy이면 data[90]로 재시도
data[93] = {7'd0, 5'd31, 5'd29, `STORE_SB, 5'd0, `OPCODE_STORE};
// SB: mem[x29+0] = 'B'

// 'E' 전송
data[94] = {12'h045, 5'd0, `ITYPE_ADDI, 5'd31, `OPCODE_ITYPE};
// ADDI: x31 = 'E' (0x45)
data[95] = {12'd0, 5'd28, `LOAD_LW, 5'd30, `OPCODE_LOAD};
// LW x30, 0(x28): Status 레지스터 읽기
data[96] = {12'd1, 5'd30, `ITYPE_ANDI, 5'd30, `OPCODE_ITYPE};
// ANDI x30, x30, 1: busy bit 마스크
data[97] = {1'b1, 6'b111111, 5'd0, 5'd30, `BRANCH_BNE, 4'b1100, 1'b1,
`OPCODE_BRANCH}; // BNE x30, x0, -8: busy이면 data[95]로 재시도
data[98] = {7'd0, 5'd31, 5'd29, `STORE_SB, 5'd0, `OPCODE_STORE};
// SB: mem[x29+0] = 'E'

data[99] = {1'b1, 10'b1110101000, 1'b1, 8'b11111111, 5'd0, `OPCODE_JAL};
// JAL x0, -176: data[55]로 분기 (EBREAK로 돌아가기)

// -----
// Trap Handler 시작 주소. mtvec = 0000_1000 = 4096 ÷ 4 Byte = 1024
// Trap Handler 진입 시 기존 GPR의 레지스터 내용들을 별도의 메모리 Heap 구역에
store하고 수행해야하지만, 현재 단계에서는 생략함.
// CSR mcause 확인해서 ecall이면 x1 = 0000_0000으로 만들기, misaligned면 x2에
FF더하기

// 조건 분기; 비교문 작성을 위한 적재 작업
data[1024] = {12'h342, 5'd0, 3'b010, 5'd6, `OPCODE_ENVIRONMENT};
// csrrs x6, mcause, x0: 레지스터 x6에 mcause값 적재
data[1025] = {12'd11, 5'd0, `ITYPE_ADDI, 5'd7, `OPCODE_ITYPE};
// addi x7, x0, 11: 레지스터 x7에 ECALL 코드 값 11 적재
data[1026] = {12'd2, 5'd0, `ITYPE_ADDI, 5'd8, `OPCODE_ITYPE};
// addi x8, x0, 2: 레지스터 x8에 ILLEGAL INSTRUCTION
코드 값 2 적재
data[1027] = {12'd4, 5'd0, `ITYPE_ADDI, 5'd9, `OPCODE_ITYPE};
// addi x9, x0, 4: 레지스터 x9에 MISALIGNED LOAD 코드 값
4 적재
data[1028] = {12'd6, 5'd0, `ITYPE_ADDI, 5'd10, `OPCODE_ITYPE};
// addi x10, x0, 6: 레지스터 x10에 MISALIGNED STORE 코드 값 6
적재

```

```

        // mcause 분석해서 해당하는 Trap Handler 주소로 분기
        data[1029] = {1'b0, 6'b0, 5'd7, 5'd6, `BRANCH_BEQ, 4'b1100, 1'b0, `OPCODE_BRANCH};
    // beq x6, x7, +24:          ECALL: x6과 x7이 같다면 24바이트 이후 주솟값으로 분기 =
data[1035]
        data[1030] = {1'b0, 6'd0, 5'd0, 5'd6, `BRANCH_BEQ, 4'b1110, 1'b0, `OPCODE_BRANCH};
    // beq x6, x0, +28:          MISALIGNED INSTRUCTION: x6값이 0과 같다면 28바이트 이후
주솟값으로 분기 = data[1037]
        data[1031] = {1'b0, 6'd0, 5'd10, 5'd6, `BRANCH_BEQ, 4'b1100, 1'b0,
`OPCODE_BRANCH};    // beq x6, x10, +24:          MISALIGNED STORE: x6값이 x10과 같다면
24바이트 이후 주솟값으로 분기 = data[1037]
        data[1032] = {1'b0, 6'd0, 5'd9, 5'd6, `BRANCH_BEQ, 4'b1010, 1'b0, `OPCODE_BRANCH};
    // beq x6, x9, +20:          MISALIGNED LOAD: x6값이 x9와 같다면 20바이트 이후 주솟값으로
분기 = data[1037]
        data[1033] = {1'b0, 6'd0, 5'd8, 5'd6, `BRANCH_BEQ, 4'b1000, 1'b0, `OPCODE_BRANCH};
    // beq x6, x8, +16:          ILLEGAL: x6값이 x8과 같다면 16바이트 이후 주솟값으로 분기 =
data[1037]
        data[1034] = {1'b0, 10'b000_0001_000, 1'b0, 8'b0, 5'd0, `OPCODE_JAL};
    // jal x0, +16:              TH 끝내기 (mret 명령어 주소로 가기)

    // ECALL Trap Handler @ data[1035]
    data[1035] = {12'd0, 5'd0, `ITYPE_ADDI, 5'd1, `OPCODE_ITYPE};
        // addi x1, x0, 0:          레지스터 x1 값 0으로 비우기
    data[1036] = {1'b0, 10'b000_0000_100, 1'b0, 8'b0, 5'd0, `OPCODE_JAL};
    // jal x0, +8:              TH 끝내기 (mret 명령어 주소로 가기)

    // ILLEGAL / MISALIGNED Trap Handler @ data[1037]
    data[1037] = {12'hFF, 5'd2, `ITYPE_ADDI, 5'd30, `OPCODE_ITYPE};
        // addi x30, x2, 255:          x30 레지스터에 x2(BC00_0000) + 0xFF =
bc00_00ff

    // ESCAPE Trap Handler @ data[1038]
    data[1038] = {12'b001100000010, 5'b0, 3'b0, 5'b0, `OPCODE_ENVIRONMENT};
    // MRET: PC = CSR[mepc]

    // HINT; NOP for 'x' signal after MRET in pipeline
    data[1039] = {12'h2BC, 5'd0, `ITYPE_ADDI, 5'd0, `OPCODE_ITYPE};
    // ADDI: x0 = x0 + 2BC = 0000_0000
    data[1040] = {12'h2BC, 5'd0, `ITYPE_ADDI, 5'd0, `OPCODE_ITYPE};
    // ADDI: x0 = x0 + 2BC = 0000_0000
    data[1041] = {12'h2BC, 5'd0, `ITYPE_ADDI, 5'd0, `OPCODE_ITYPE};
    // ADDI: x0 = x0 + 2BC = 0000_0000
    data[1042] = {12'h2BC, 5'd0, `ITYPE_ADDI, 5'd0, `OPCODE_ITYPE};
    // ADDI: x0 = x0 + 2BC = 0000_0000

end

always @(*) begin
    instruction = data[pc[31:2]];
end

wire rom_access = (rom_address[31:16] == 16'h0000);
always @(*) begin

```

```

        if (rom_access) begin
            rom_read_data = data[rom_address[15:2]];
        end else begin
            rom_read_data = 32'b0;
        end
    end
end
endmodule

```

MEM_WB_Register.v

```

module MEM_WB_Register #(
    parameter XLEN = 32
)(
    // pipeline register control signals
    input wire clk,
    input wire reset,
    input wire MEM_WB_stall,
    input wire flush,

    // signals from EX/MEM register
    input wire [XLEN-1:0] MEM_pc,
    input wire [XLEN-1:0] MEM_pc_plus_4,
    input wire [31:0] MEM_instruction,

    input wire [2:0] MEM_register_file_write_data_select,
    input wire [XLEN-1:0] MEM_imm,
    input wire [19:0] MEM_raw_imm,
    input wire [XLEN-1:0] MEM_csr_read_data,
    input wire [XLEN-1:0] MEM_alu_result,
    input wire MEM_register_write_enable,
    input wire MEM_csr_write_enable,
    input wire [4:0] MEM_rs1,
    input wire [4:0] MEM_rd,
    input wire [6:0] MEM_opcode,

    // signals from MEM phase
    input wire [XLEN-1:0] MEM_byte_enable_logic_register_file_write_data,

    // signals to MEM register
    output reg [XLEN-1:0] WB_pc,
    output reg [XLEN-1:0] WB_pc_plus_4,
    output reg [31:0] WB_instruction,

    output reg [2:0] WB_register_file_write_data_select,
    output reg [XLEN-1:0] WB_imm,
    output reg [19:0] WB_raw_imm,
    output reg [XLEN-1:0] WB_csr_read_data,
    output reg [XLEN-1:0] WB_alu_result,
    output reg WB_register_write_enable,
    output reg WB_csr_write_enable,
    output reg [4:0] WB_rs1,
    output reg [4:0] WB_rd,
    output reg [6:0] WB_opcode,

```

```

    output reg [XLEN-1:0] WB_byte_enable_logic_register_file_write_data
);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        WB_pc <= {XLEN{1'b0}};
        WB_pc_plus_4 <= {XLEN{1'b0}};
        WB_instruction <= 32'h0000_0013; // ADDI x0, x0, 0 = RISC-V NOP, HINT

        WB_register_file_write_data_select <= 3'b0;
        WB_imm <= {XLEN{1'b0}};
        WB_raw_imm <= 20'b0;
        WB_csr_read_data <= {XLEN{1'b0}};
        WB_alu_result <= {XLEN{1'b0}};
        WB_register_write_enable <= 1'b0;
        WB_csr_write_enable <= 1'b0;
        WB_rs1 <= 5'b0;
        WB_rd <= 5'b0;
        WB_opcode <= 7'b0;

        WB_byte_enable_logic_register_file_write_data <= {XLEN{1'b0}};
    end else begin
        if (flush) begin
            WB_pc <= {XLEN{1'b0}};
            WB_pc_plus_4 <= {XLEN{1'b0}};
            WB_instruction <= 32'h0000_0013; // ADDI x0, x0, 0 = RISC-V NOP, HINT

            WB_register_file_write_data_select <= 3'b0;
            WB_imm <= {XLEN{1'b0}};
            WB_raw_imm <= 20'b0;
            WB_csr_read_data <= {XLEN{1'b0}};
            WB_alu_result <= {XLEN{1'b0}};
            WB_register_write_enable <= 1'b0;
            WB_csr_write_enable <= 1'b0;
            WB_rs1 <= 5'b0;
            WB_rd <= 5'b0;
            WB_opcode <= 7'b0;

            WB_byte_enable_logic_register_file_write_data <= {XLEN{1'b0}};
        end else if (!MEM_WB_stall) begin
            WB_pc <= MEM_pc;
            WB_pc_plus_4 <= MEM_pc_plus_4;
            WB_instruction <= MEM_instruction;

            WB_register_file_write_data_select <= MEM_register_file_write_data_select;
            WB_imm <= MEM_imm;
            WB_raw_imm <= MEM_raw_imm;
            WB_csr_read_data <= MEM_csr_read_data;
            WB_alu_result <= MEM_alu_result;
            WB_register_write_enable <= MEM_register_write_enable;
            WB_csr_write_enable <= MEM_csr_write_enable;
            WB_rs1 <= MEM_rs1;
            WB_rd <= MEM_rd;
            WB_opcode <= MEM_opcode;
        end
    end
end

```

```

        WB_byte_enable_logic_register_file_write_data <= MEM_byte_enable_logic_register_file_write_data;
    end
end
end

endmodule

```

Register_File.v

```

module RegisterFile (
    input clk,           // clock signal
    input [4:0] read_reg1, // take address of register 1 to read stored value
    input [4:0] read_reg2, // take address of register 2 to read stored value
    input [4:0] write_reg, // take address of register to write value
    input [31:0] write_data, // data to write
    input write_enable,    // enabling signal for writing register

    output reg [31:0] read_data1, // data from register 1
    output reg [31:0] read_data2 // data from register 2
);

reg [31:0] registers [0:31]; // 32 registers with 32 bits each

integer i;
initial begin
    for (i = 1; i < 32; i = i + 1) registers[i] = 32'b0;
end

// Read operation
always @(*) begin
    if (read_reg1 == 5'd0) begin
        read_data1 = 32'd0;
    end

    else if (write_enable && (write_reg == read_reg1)) begin
        read_data1 = write_data;
    end

    else begin
        read_data1 = registers[read_reg1];
    end

    if (read_reg2 == 5'd0) begin
        read_data2 = 32'd0;
    end

    else if (write_enable && (write_reg == read_reg2)) begin
        read_data2 = write_data;
    end

    else begin
        read_data2 = registers[read_reg2];
    end
end
end

```

```

// Write operation
always @(posedge clk) begin
    if (write_enable && write_reg != 5'd0) begin
        registers[write_reg] <= write_data; // write to register if not x0
    end
end

endmodule

```

ID_EX_Register.v

```

module ID_EX_Register #(
    parameter XLEN = 32
)(
    // pipeline register control signals
    input wire clk,
    input wire reset,
    input wire flush,
    input wire ID_EX_stall,

    // signal from IF phase
    // input wire [XLEN-1:0] IF_PC but why? What was this for?

    // signals from IF/ID register
    input wire [XLEN-1:0] ID_pc,
    input wire [XLEN-1:0] ID_pc_plus_4,
    input wire ID_branch_estimation,
    input wire [31:0] ID_instruction,

    // signals from ID phase
    input wire ID_jump,
    input wire ID_branch,
    input wire [1:0] ID_alu_src_A_select,
    input wire [2:0] ID_alu_src_B_select,
    input wire ID_memory_read,
    input wire ID_memory_write,
    input wire [2:0] ID_register_file_write_data_select,
    input wire ID_register_write_enable,
    input wire ID_csr_write_enable,
    input wire [6:0] ID_opcode,
    input wire [2:0] ID_funct3,
    input wire [6:0] ID_funct7,
    input wire [4:0] ID_rd,
    input wire [19:0] ID_raw_imm,
    input wire [XLEN-1:0] ID_read_data1,
    input wire [XLEN-1:0] ID_read_data2,
    input wire [4:0] ID_rs1,
    input wire [4:0] ID_rs2,
    input wire [XLEN-1:0] ID_imm,
    input wire [XLEN-1:0] ID_csr_read_data,

    // signals to EX/MEM register
    output reg [XLEN-1:0] EX_pc,
    output reg [XLEN-1:0] EX_pc_plus_4,
    output reg EX_branch_estimation,

```



```

output reg [31:0] EX_instruction,

output reg EX_jump,
output reg EX_memory_read,
output reg EX_memory_write,
output reg [2:0] EX_register_file_write_data_select,
output reg EX_register_write_enable,
output reg EX_csr_write_enable,
output reg EX_branch,
output reg [1:0] EX_alu_src_A_select,
output reg [2:0] EX_alu_src_B_select,
output reg [6:0] EX_opcode,
output reg [2:0] EX_funct3,
output reg [6:0] EX_funct7,
output reg [4:0] EX_rd,
output reg [19:0] EX_raw_imm,
output reg [XLEN-1:0] EX_read_data1,
output reg [XLEN-1:0] EX_read_data2,
output reg [4:0] EX_rs1,
output reg [4:0] EX_rs2,
output reg [XLEN-1:0] EX_imm,
output reg [XLEN-1:0] EX_csr_read_data
);

always @(posedge clk or posedge reset) begin
  if (reset) begin
    EX_pc <= {XLEN{1'b0}};
    EX_pc_plus_4 <= {XLEN{1'b0}};
    EX_branch_estimation <= 1'b0;
    EX_instruction <= 32'h0000_0013; // ADDI x0, x0, 0 = RISC-V NOP, HINT

    EX_jump <= 1'b0;
    EX_memory_read <= 1'b0;
    EX_memory_write <= 1'b0;
    EX_register_file_write_data_select <= 3'b0;
    EX_register_write_enable <= 1'b0;
    EX_csr_write_enable <= 1'b0;
    EX_branch <= 1'b0;
    EX_alu_src_A_select <= 2'b0;
    EX_alu_src_B_select <= 3'b0;
    EX_opcode <= 7'b0;
    EX_funct3 <= 3'b0;
    EX_funct7 <= 7'b0;
    EX_rd <= 5'b0;
    EX_raw_imm <= 20'b0;
    EX_read_data1 <= {XLEN{1'b0}};
    EX_read_data2 <= {XLEN{1'b0}};
    EX_rs1 <= 5'b0;
    EX_rs2 <= 5'b0;
    EX_imm <= {XLEN{1'b0}};
    EX_csr_read_data <= {XLEN{1'b0}};
  end else begin
    if (flush) begin
      EX_pc <= {XLEN{1'b0}};
    end
  end
end

```

```

EX_pc_plus_4 <= {XLEN{1'b0}};
EX_branch_estimation <= 1'b0;
EX_instruction <= 32'h0000_0013; // ADDI x0, x0, 0 = RISC-V NOP, HINT

EX_jump <= 1'b0;
EX_memory_read <= 1'b0;
EX_memory_write <= 1'b0;
EX_register_file_write_data_select <= 3'b0;
EX_register_write_enable <= 1'b0;
EX_csr_write_enable <= 1'b0;
EX_branch <= 1'b0;
EX_alu_src_A_select <= 2'b0;
EX_alu_src_B_select <= 3'b0;
EX_opcode <= 7'b0;
EX_funct3 <= 3'b0;
EX_funct7 <= 7'b0;
EX_rd <= 5'b0;
EX_raw_imm <= 20'b0;
EX_read_data1 <= {XLEN{1'b0}};
EX_read_data2 <= {XLEN{1'b0}};
EX_rs1 <= 5'b0;
EX_rs2 <= 5'b0;
EX_imm <= {XLEN{1'b0}};
EX_csr_read_data <= {XLEN{1'b0}};
end else if (!ID_EX_stall) begin
    EX_pc <= ID_pc;
    EX_pc_plus_4 <= ID_pc_plus_4;
    EX_branch_estimation <= ID_branch_estimation;
    EX_instruction <= ID_instruction;

    EX_jump <= ID_jump;
    EX_memory_read <= ID_memory_read;
    EX_memory_write <= ID_memory_write;
    EX_register_file_write_data_select <= ID_register_file_write_data_select;
    EX_register_write_enable <= ID_register_write_enable;
    EX_csr_write_enable <= ID_csr_write_enable;
    EX_branch <= ID_branch;
    EX_alu_src_A_select <= ID_alu_src_A_select;
    EX_alu_src_B_select <= ID_alu_src_B_select;
    EX_opcode <= ID_opcode;
    EX_funct3 <= ID_funct3;
    EX_funct7 <= ID_funct7;
    EX_rd <= ID_rd;
    EX_raw_imm <= ID_raw_imm;
    EX_read_data1 <= ID_read_data1;
    EX_read_data2 <= ID_read_data2;
    EX_rs1 <= ID_rs1;
    EX_rs2 <= ID_rs2;
    EX_imm <= ID_imm;
    EX_csr_read_data <= ID_csr_read_data;
end
end
end

```

```
endmodule
```

Immediate_Generator.v

```
`include "modules/headers/opcode.vh"
```

```
module ImmediateGenerator (
    input [19:0] raw_imm,    // raw immediate value from Instruction Decoder
    input [6:0] opcode,      // opcode from Instruction Decoder
    output reg [31:0] imm    // sign-extension of the raw immediate value
);

    always @(*) begin
        case (opcode)
            `OPCODE_JALR, `OPCODE_LOAD, `OPCODE_ITYPE, `OPCODE_FENCE,
            `OPCODE_ENVIRONMENT: begin // I-type
                imm = {{20{raw_imm[11]}}, raw_imm[11:0]};
            end
            `OPCODE_STORE: begin // S-Type
                imm = {{20{raw_imm[11]}}, raw_imm[11:0]};
            end
            `OPCODE_LUI, `OPCODE_AUIPC: begin // U-Type
                imm = {raw_imm, 12'b0};
            end
            `OPCODE_BRANCH: begin // B-Type
                imm = {{19{raw_imm[11]}}, raw_imm[11:0], 1'b0};
            end
            `OPCODE_JAL: begin // J-Type
                imm = {{11{raw_imm[19]}}, raw_imm[19:0], 1'b0};
            end
            default: begin
                imm = 32'b0;
            end
        endcase
    end

endmodule
```

Instruction_Decoder.v

```
`include "modules/headers/opcode.vh"
```

```
module InstructionDecoder (
    input [31:0] instruction,

    output reg [6:0] opcode,
    output reg [2:0] funct3,
    output reg [6:0] funct7,
    output reg [4:0] rs1,
    output reg [4:0] rs2,
    output reg [4:0] rd,
    output reg [19:0] raw_imm
);

    wire [6:0] opcode_wire = instruction[6:0];
```

```

always @(*) begin
    opcode = opcode_wire[6:0];
    case (opcode)
        `OPCODE_LUI, `OPCODE_AUIPC: begin // U-type
            rd = instruction[11:7];
            raw_imm = instruction[31:12];

            funct3 = 3'b000;
            rs1 = 5'b00000;
            rs2 = 5'b00000;
            funct7 = 7'b0000000;

        end

        `OPCODE_JAL: begin // J-type
            rd = instruction[11:7];
            raw_imm = {instruction[31], instruction[19:12], instruction[20], instruction[30:21]};

            funct3 = 3'b000;
            rs1 = 5'b00000;
            rs2 = 5'b00000;
            funct7 = 7'b0000000;

        end

        `OPCODE_JALR, `OPCODE_LOAD, `OPCODE_ITYPE, `OPCODE_FENCE,
        `OPCODE_ENVIRONMENT: begin // I-type
            rd = instruction[11:7];
            funct3 = instruction[14:12];
            rs1 = instruction[19:15];
            raw_imm = {8'b0, instruction[31:20]};

            rs2 = 5'b00000;
            funct7 = 7'b0000000;

        end

        `OPCODE_BRANCH: begin // B-type
            funct3 = instruction[14:12];
            rs1 = instruction[19:15];
            rs2 = instruction[24:20];
            raw_imm = {instruction[31], instruction[7], instruction[30:25], instruction[11:8]};

            rd = 5'b00000;
            funct7 = 7'b0000000;

        end

        `OPCODE_STORE: begin // S-type
            funct3 = instruction[14:12];
            rs1 = instruction[19:15];
            rs2 = instruction[24:20];
            raw_imm = {8'b0, instruction[31:25], instruction[11:7]};

            rd = 5'b00000;
            funct7 = 7'b0000000;

        end
    end
end

```

```

        `OPCODE_RTYPE: begin // R type
            rd = instruction[11:7];
            funct3 = instruction[14:12];
            rs1 = instruction[19:15];
            rs2 = instruction[24:20];
            funct7 = instruction[31:25];

            raw_imm = 32'b0;
        end
        default: begin
            funct3 = 3'b0;

            funct7 = 7'b0;
            rs1 = 5'b0;
            rs2 = 5'b0;
            rd = 5'b0;
            raw_imm = 20'b0;
        end
    endcase
end

endmodule

```

Instruction_Cache.v

```

module InstructionCache (
    input clk,
    input reset,
    input [9:0] address,
    input update_enable,
    input [31:0] im_data,

    output reg hit,
    output reg [31:0] data
);

    reg valid1 [0:31];
    reg [4:0] tag1 [0:31];
    reg [31:0] data1 [0:31];

    reg valid2 [0:31];
    reg [4:0] tag2 [0:31];
    reg [31:0] data2 [0:31];

    reg lru [0:31];

    wire [4:0] index = address[4:0];
    wire [4:0] tag = address[9:5];

    wire hit1 = valid1[index] && (tag1[index] == tag);
    wire hit2 = valid2[index] && (tag2[index] == tag);

    always @(*) begin
        hit = hit1 || hit2;
    end
end

```

```

integer i;
always @(posedge clk or posedge reset) begin
    if (reset) begin
        for (i=0; i<32; i=i+1) begin
            valid1[i] <= 0;
            tag1[i] <= 5'b0;
            data1[i] <= 32'b0;

            valid2[i] <= 0;
            tag2[i] <= 5'b0;
            data2[i] <= 32'b0;

            lru[i] <= 0;
        end

        data <= 32'b0;
    end
    else begin
        if (hit1) begin
            data <= data1[index];
        end
        else if (hit2) begin
            data <= data2[index];
        end
        else begin
            if (update_enable) begin
                if (!valid1[index]) begin
                    valid1[index] <= 1;
                    tag1[index] <= tag;
                    data1[index] <= im_data;
                end
                else if (!valid2[index]) begin
                    valid2[index] <= 1;
                    tag2[index] <= tag;
                    data2[index] <= im_data;
                end
                else if (lru[index] == 0) begin
                    tag1[index] <= tag;
                    data1[index] <= im_data;
                    lru[index] <= 1;
                end
                else begin
                    tag2[index] <= tag;
                    data2[index] <= im_data;
                    lru[index] <= 0;
                end
            end
            data <= 32'b0;
        end
    end
end
endmodule

```

Miscellaneous

Branch_Predictor.v

```
`include "modules/headers/opcode.vh"

module BranchPredictor #(
    parameter XLEN = 32
) (
    input wire clk,
    input wire reset,
    input wire [6:0] IF_opcode,
    input wire [XLEN-1:0] IF_pc,
    input wire [XLEN-1:0] IF_imm,
    input wire EX_branch,
    input wire EX_branch_taken,

    output reg branch_estimation,
    output reg [XLEN-1:0] branch_target
);
    reg [1:0] prediction_counter;
    reg branch_prediction;
    wire [XLEN-1:0] prediction_target = branch_estimation ? (IF_pc + IF_imm) : (IF_pc + {{XLEN-3{1'b0}},3'd4});

    always @(*) begin
        if (IF_opcode == `OPCODE_BRANCH) begin
            branch_estimation = prediction_counter[1];
            branch_target = prediction_target;
        end
        else begin
            branch_estimation = 1'b0;
            branch_target = {XLEN{1'b0}};
        end
    end
end

always @(posedge clk or posedge reset) begin
    if (reset) begin
        branch_prediction <= 1'b0;
        prediction_counter <= 2'b00;
    end else begin
        if (IF_opcode == `OPCODE_BRANCH) begin
            branch_prediction <= branch_estimation;
        end
    end

    if (EX_branch && (EX_branch_taken == 1'b1 || EX_branch_taken == 1'b0)) begin
```

```

        case ({EX_branch_taken, prediction_counter})
        // Not Taken 0, prediction_counter 00
        3'b0_00: prediction_counter <= 2'b00; // Strongly Not Taken <- if not taken again, still 00.
        3'b0_01: prediction_counter <= 2'b00; // Weakly Not Taken <- if not taken again, 00.
        3'b0_10: prediction_counter <= 2'b01; // Weakly Taken <- if not taken, 01.
        3'b0_11: prediction_counter <= 2'b10; // Strongly Taken <- if not taken, 10.
        // Taken 1, prediction_counter 00
        3'b1_00: prediction_counter <= 2'b01; // Strongly Not Taken <- if taken, 01.
        3'b1_01: prediction_counter <= 2'b10; // Weakly Not Taken <- if taken, 10.
        3'b1_10: prediction_counter <= 2'b11; // Weakly Taken <- if taken again, 11.
        3'b1_11: prediction_counter <= 2'b11; // Strongly Taken <- if taken again, still 11.
    endcase
end
end
end

```

```
endmodule
```

Hazard_Unit.v

```

`include "modules/headers/opcode.vh"
`include "modules/headers/trap.vh"

module HazardUnit (
    input clk,
    input reset,

    input wire trap_done,
    input wire csr_ready,
    input wire standby_mode,
    input wire [2:0] trap_status,
    input wire misaligned_instruction_flush,
    input wire misaligned_memory_flush,
    input wire pth_done_flush,

    input wire [4:0] ID_rs1,
    input wire [4:0] ID_rs2,
    input wire [11:0] ID_raw_imm,

    input wire [4:0] MEM_rd,
    input wire MEM_register_write_enable,
    input wire MEM_csr_write_enable,
    input wire [11:0] MEM_csr_write_address,    // MEM_imm[11:0]

    input wire [4:0] WB_rd,
    input wire WB_register_write_enable,
    input wire WB_csr_write_enable,
    input wire [11:0] WB_csr_write_address, // WB_imm[11:0]

    input wire [4:0] EX_rd,
    input wire [6:0] EX_opcode,
    input wire [4:0] EX_rs1,

```



```

input wire [4:0] EX_rs2,
input wire [11:0] EX_imm, // EX_imm[11:0]

input wire EX_csr_write_enable,

input wire EX_jump,
input wire branch_prediction_miss,

// to Forward Unit - ALU forwarding
output reg [1:0] hazard_mem,
output reg [1:0] hazard_wb,
output wire csr_hazard_mem,
output wire csr_hazard_wb,
//output reg csr_reg_hazard,

/// to Forward Unit - Store data forwarding
output wire store_hazard_mem,
output wire store_hazard_wb,

output reg IF_ID_flush,
output reg ID_EX_flush,
output reg EX_MEM_flush,
output reg MEM_WB_flush,

output reg IF_ID_stall,
output reg ID_EX_stall,
output reg EX_MEM_stall,
output reg MEM_WB_stall
);

// Store instruction detection
wire is_store = (EX_opcode == `OPCODE_STORE);

// Store instruction rs2 hazard detections
assign store_hazard_mem = is_store && mem_hazard_rs2;
assign store_hazard_wb = is_store && wb_hazard_rs2 && !mem_hazard_rs2;

// wire reg_csr_hazard = (EX_opcode == `OPCODE_ENVIRONMENT && (WB_rd == retire_rd) &&
(WB_csr_write_address == retire_csr_write_address));

// Register ALU hazard detections
wire mem_hazard_rs1 = MEM_register_write_enable && (MEM_rd != 5'd0) && (MEM_rd == EX_rs1);
wire mem_hazard_rs2 = MEM_register_write_enable && (MEM_rd != 5'd0) && (MEM_rd == EX_rs2);
wire wb_hazard_rs1 = WB_register_write_enable && (WB_rd != 5'd0) && (WB_rd == EX_rs1);
wire wb_hazard_rs2 = WB_register_write_enable && (WB_rd != 5'd0) && (WB_rd == EX_rs2);

// CSR hazard detection
assign csr_hazard_mem = MEM_csr_write_enable && (MEM_csr_write_address == EX_imm);
assign csr_hazard_wb = WB_csr_write_enable && (WB_csr_write_address == EX_imm);

reg [4:0] retire_rd;
reg [11:0] retire_csr_write_address;

always @(posedge clk or posedge reset) begin

```

```

    if (reset) begin
        retire_rd <= 5'b0;
        retire_csr_write_address <= 12'b0;
    end else begin
        retire_rd <= WB_rd;
        retire_csr_write_address <= WB_csr_write_address;
    end

end

always @(*) begin
    //csr_reg_hazard = 1'b0;
    hazard_mem = 2'b00;
    hazard_wb = 2'b00;
    IF_ID_flush = 1'b0;
    ID_EX_flush = 1'b0;
    EX_MEM_flush = 1'b0;
    MEM_WB_flush = 1'b0;

    IF_ID_stall = 1'b0;
    ID_EX_stall = 1'b0;
    EX_MEM_stall = 1'b0;
    MEM_WB_stall = 1'b0;

    // ALU forwarding hazards
    // For Store instructions, rs2 hazard shouldn't trigger ALUsrcB forwarding.
    // In this case, rs2 is store data, not ALU operand.
    hazard_mem[0] = mem_hazard_rs1;
    hazard_mem[1] = is_store ? 1'b0 : mem_hazard_rs2; // Disables ALUsrcB forwarding for store
    hazard_wb[0] = wb_hazard_rs1 && !mem_hazard_rs1;
    hazard_wb[1] = is_store ? 1'b0 : (wb_hazard_rs2 && !mem_hazard_rs2); // Disables ALUsrcB forwarding for store

    /*if (reg_csr_hazard) begin
        csr_reg_hazard = 1'b1;
    end*/

    if (trap_done && (branch_prediction_miss || EX_jump)) begin
        IF_ID_flush = 1'b1;
        ID_EX_flush = 1'b1;
    end

    if (pth_done_flush) begin
        IF_ID_flush = 1'b1;
        ID_EX_flush = 1'b1;
        EX_MEM_flush = 1'b1;
        MEM_WB_flush = 1'b1;
    end

    if (standby_mode) begin // For ID Phase Exception handling
        IF_ID_stall = 1'b1;
        ID_EX_stall = 1'b1;
        EX_MEM_stall = 1'b0;
        MEM_WB_stall = 1'b0;
    end else if (!trap_done || !csr_ready) begin

```

```

        IF_ID_stall = 1'b1;
        ID_EX_stall = 1'b1;
        EX_MEM_stall = 1'b1;
        MEM_WB_stall = 1'b1;
    end
end

```

```
endmodule
```

Forward_Unit.v

```

`include "modules/headers/opcode.vh"
`include "modules/headers/trap.vh"

module HazardUnit (
    input clk,
    input reset,

    input wire trap_done,
    input wire csr_ready,
    input wire standby_mode,
    input wire [2:0] trap_status,
    input wire misaligned_instruction_flush,
    input wire misaligned_memory_flush,
    input wire pth_done_flush,

    input wire [4:0] ID_rs1,
    input wire [4:0] ID_rs2,
    input wire [11:0] ID_raw_imm,

    input wire [4:0] MEM_rd,
    input wire MEM_register_write_enable,
    input wire MEM_csr_write_enable,
    input wire [11:0] MEM_csr_write_address,    // MEM_imm[11:0]

    input wire [4:0] WB_rd,
    input wire WB_register_write_enable,
    input wire WB_csr_write_enable,
    input wire [11:0] WB_csr_write_address, // WB_imm[11:0]

    input wire [4:0] EX_rd,
    input wire [6:0] EX_opcode,
    input wire [4:0] EX_rs1,
    input wire [4:0] EX_rs2,
    input wire [11:0] EX_imm, // EX_imm[11:0]

    input wire EX_csr_write_enable,

    input wire EX_jump,
    input wire branch_prediction_miss,

```

```

// to Forward Unit - ALU forwarding
output reg [1:0] hazard_mem,
output reg [1:0] hazard_wb,
output wire csr_hazard_mem,
output wire csr_hazard_wb,
//output reg csr_reg_hazard,

/// to Forward Unit - Store data forwarding
output wire store_hazard_mem,
output wire store_hazard_wb,

output reg IF_ID_flush,
output reg ID_EX_flush,
output reg EX_MEM_flush,
output reg MEM_WB_flush,

output reg IF_ID_stall,
output reg ID_EX_stall,
output reg EX_MEM_stall,
output reg MEM_WB_stall
);

// Store instruction detection
wire is_store = (EX_opcode == `OPCODE_STORE);

// Store instruction rs2 hazard detections
assign store_hazard_mem = is_store && mem_hazard_rs2;
assign store_hazard_wb = is_store && wb_hazard_rs2 && !mem_hazard_rs2;

// wire reg_csr_hazard = (EX_opcode == `OPCODE_ENVIRONMENT && (WB_rd == retire_rd) &&
(WB_csr_write_address == retire_csr_write_address));

// Register ALU hazard detections
wire mem_hazard_rs1 = MEM_register_write_enable && (MEM_rd != 5'd0) && (MEM_rd == EX_rs1);
wire mem_hazard_rs2 = MEM_register_write_enable && (MEM_rd != 5'd0) && (MEM_rd == EX_rs2);
wire wb_hazard_rs1 = WB_register_write_enable && (WB_rd != 5'd0) && (WB_rd == EX_rs1);
wire wb_hazard_rs2 = WB_register_write_enable && (WB_rd != 5'd0) && (WB_rd == EX_rs2);

// CSR hazard detection
assign csr_hazard_mem = MEM_csr_write_enable && (MEM_csr_write_address == EX_imm);
assign csr_hazard_wb = WB_csr_write_enable && (WB_csr_write_address == EX_imm);

reg [4:0] retire_rd;
reg [11:0] retire_csr_write_address;

always @(posedge clk or posedge reset) begin
    if (reset) begin
        retire_rd <= 5'b0;
        retire_csr_write_address <= 12'b0;
    end else begin
        retire_rd <= WB_rd;
        retire_csr_write_address <= WB_csr_write_address;
    end
end

```

end

always @(*) begin

 //csr_reg_hazard = 1'b0;

 hazard_mem = 2'b00;

 hazard_wb = 2'b00;

 IF_ID_flush = 1'b0;

 ID_EX_flush = 1'b0;

 EX_MEM_flush = 1'b0;

 MEM_WB_flush = 1'b0;

 IF_ID_stall = 1'b0;

 ID_EX_stall = 1'b0;

 EX_MEM_stall = 1'b0;

 MEM_WB_stall = 1'b0;

 // ALU forwarding hazards

 // For Store instructions, rs2 hazard shouldn't trigger ALUsrcB forwarding.

 // In this case, rs2 is store data, not ALU operand.

 hazard_mem[0] = mem_hazard_rs1;

 hazard_mem[1] = is_store ? 1'b0 : mem_hazard_rs2; // Disables ALUsrcB forwarding for store

 hazard_wb[0] = wb_hazard_rs1 && !mem_hazard_rs1;

 hazard_wb[1] = is_store ? 1'b0 : (wb_hazard_rs2 && !mem_hazard_rs2); // Disables ALUsrcB forwarding for store

 /*if (reg_csr_hazard) begin

 csr_reg_hazard = 1'b1;

 end*/

 if (trap_done && (branch_prediction_miss || EX_jump)) begin

 IF_ID_flush = 1'b1;

 ID_EX_flush = 1'b1;

 end

 if (pth_done_flush) begin

 IF_ID_flush = 1'b1;

 ID_EX_flush = 1'b1;

 EX_MEM_flush = 1'b1;

 MEM_WB_flush = 1'b1;

 end

 if (standby_mode) begin // For ID Phase Exception handling

 IF_ID_stall = 1'b1;

 ID_EX_stall = 1'b1;

 EX_MEM_stall = 1'b0;

 MEM_WB_stall = 1'b0;

 end else if (!trap_done || !csr_ready) begin

 IF_ID_stall = 1'b1;

 ID_EX_stall = 1'b1;

 EX_MEM_stall = 1'b1;

 MEM_WB_stall = 1'b1;

 end

end

endmodule

CSR_File.v

```
`include "modules/headers/csr.vh"
```

```
module CSRFile #(
    parameter XLEN = 32
)(
    input clk,                // clock signal
    input reset,              // reset signal
    input trapped,
    input csr_write_enable,    // write enable signal
    input [11:0] csr_read_address, // address to read
    input [11:0] csr_write_address, // address to write
    input [XLEN-1:0] csr_write_data, // data to write
    input instruction_retired,

    output reg [XLEN-1:0] csr_read_out, // data from CSR Unit
    output reg csr_ready // signal to stall the process while accessing the CSR until it outputs the desired value.
);

wire [XLEN-1:0] mvendorid = 32'h52_56_4B_43; // "RVKC" ; "R"ISC-"V", "K"HWL & "C"hoiCube84.
wire [XLEN-1:0] marchid = 32'h34_36_53_35; // "46S5" ; "46"F arch based "S"uper scalar "5"-Stage Pipeline Architecture.
wire [XLEN-1:0] mimpid = 32'h34_36_49_31; // "46I1" ; "46" instructions RISC-V RV32"I" Revision "1".
wire [XLEN-1:0] mhartid = 32'h52_4B_43_30; // "RKC0" ; "R"oad to "K"AIST "C"ore 0.
wire [XLEN-1:0] mstatus = 32'h00001800; // MPP[12:11] = 11
wire [XLEN-1:0] misa = 32'h40000100; // MXL = 32; misa[31:30] = 01. RV32"I"; misa[8] = 1.

reg [XLEN-1:0] mtvec;
reg [XLEN-1:0] mepc;
reg [XLEN-1:0] mcause;

reg [63:0] mcycle;
reg [63:0] minstret;

reg csr_processing;
reg [XLEN-1:0] csr_read_data;

wire csr_access;
wire valid_csr_address;

assign csr_access = valid_csr_address;
assign valid_csr_address = (csr_read_address == 12'hB00) || // mcycle
    (csr_read_address == 12'hB02) || // minstret
    (csr_read_address == 12'hB80) || // mcycleh
    (csr_read_address == 12'hB82) || // minstreth
    (csr_read_address == 12'hF11) || // mvendorid
    (csr_read_address == 12'hF12) || // marchid
    (csr_read_address == 12'hF13) || // mimpid
    (csr_read_address == 12'hF14) || // mhartid
    (csr_read_address == 12'h300) || // mstatus
    (csr_read_address == 12'h301) || // misa
```

```

        (csr_read_address == 12'h305) || // mtvec
        (csr_read_address == 12'h341) || // mepc
        (csr_read_address == 12'h342); // mcause

```

```

localparam [XLEN-1:0] DEFAULT_mtvec = 32'h00001000;
localparam [XLEN-1:0] DEFAULT_mepc = {XLEN{1'b0}};
localparam [XLEN-1:0] DEFAULT_mcause = {XLEN{1'b0}};
localparam [63:0] DEFAULT_mcycle = 64'b0;
localparam [63:0] DEFAULT_minstret = 64'b0;

```

// Read Operation.

```

always @(*) begin
    case (csr_read_address)
        12'hB00: csr_read_data = mcycle[XLEN-1:0];
        12'hB02: csr_read_data = minstret[XLEN-1:0];
        12'hB80: csr_read_data = mcycle[63:32];
        12'hB82: csr_read_data = minstret[63:32];
        12'hF11: csr_read_data = mvendorid;
        12'hF12: csr_read_data = marchid;
        12'hF13: csr_read_data = mimpid;
        12'hF14: csr_read_data = mhartid;
        12'h300: csr_read_data = mstatus;
        12'h301: csr_read_data = misa;
        12'h305: csr_read_data = mtvec;
        12'h341: csr_read_data = mepc;
        12'h342: csr_read_data = mcause;
        default: csr_read_data = {XLEN{1'b0}};
    endcase

```

```

    if (reset) begin
        csr_ready = 1'b1;
    end else begin
        if (csr_access && !csr_processing) begin
            csr_ready = 1'b0;
        end else if (csr_processing) begin
            csr_ready = 1'b1;
        end else begin
            csr_ready = 1'b1;
        end
    end
end

```

// Reset Operation

```

always @(posedge clk or posedge reset) begin
    if (reset) begin
        mtvec <= DEFAULT_mtvec;
        mepc <= DEFAULT_mepc;
        mcause <= DEFAULT_mcause;
        mcycle <= DEFAULT_mcycle;
        minstret <= DEFAULT_minstret;

        csr_processing <= 1'b0;
    end

```

```

        csr_read_out <= {XLEN{1'b0}};
    end else begin
        mcycle <= mcycle + 1;

        if (instruction_retired) begin
            minstret <= minstret + 1;
        end

        if (csr_access && !csr_processing) begin
            csr_processing <= 1'b1;
            csr_read_out <= csr_read_data;
        end else if (csr_processing) begin
            csr_processing <= 1'b0;
            csr_read_out <= csr_read_data;
        end else if (csr_write_enable) begin
            csr_read_out <= csr_read_data;
        end

        // Write Operation
        if ((trapped && csr_write_enable) || (csr_write_enable)) begin
            case (csr_write_address)
                12'h305: mtvec <= csr_write_data;
                12'h341: mepc <= csr_write_data;
                12'h342: mcause <= csr_write_data;
                default: ;
            endcase
        end
    end
end

endmodule

```

Exception_Detector.v

```

`include "modules/headers/opcode.vh"
`include "modules/headers/store.vh"
`include "modules/headers/load.vh"
`include "modules/headers/trap.vh"

module ExceptionDetector (
    input clk,
    input reset,
    input [6:0] ID_opcode,      // opcode from ID Phase
    input [6:0] EX_opcode,     // opcode from EX Phase
    input [6:0] MEM_opcode,    // opcode from MEM Phase for Synchronous Exception Detector
    // input [4:0] rs1,         // For Illegal Instruction Exception
    input [2:0] ID_funct3,     // funct3
    input [2:0] EX_funct3,
    input [2:0] MEM_funct3,
    input [1:0] alu_result,    // LSBs of jump target and data_memory_address
    input [1:0] MEM_alu_result,
    input [11:0] raw_imm,      // raw_imm field to distinguish EBREAK, ECALL and MRET, CSR_Address
    input [11:0] EX_raw_imm,

```



```

input csr_write_enable,      // CSR WE signal from Control Unit for Detecting Illegal CSR access from ID Phase.
input [1:0] branch_target_lsbs,    // LSBs of branch target
input branch_estimation,

output reg trapped,          // signal indicating if trap has occurred
output reg [2:0] trap_status // current trap status
);
reg ID_trapped;
reg [2:0] ID_trap_status;
reg EX_trapped;
reg [2:0] EX_trap_status;
reg MEM_trapped;
reg [2:0] MEM_trap_status;
reg trapped_combinatorial;
reg [2:0] trap_status_combinatorial;

/* For Illegal Instruction Exception
wire csr_write;
assign csr_write = (ID_funct3 == `CSR_CSRRW || ID_funct3 == `CSR_CSRRWI)
    || ((ID_funct3 == `CSR_CSRRS || ID_funct3 == `CSR_CSRRSI) && (rs1 != 5'b0))
    || ((ID_funct3 == `CSR_CSRRC || ID_funct3 == `CSR_CSRRCI) && (rs1 != 5'b0));

wire valid_write_csr;
assign valid_write_csr = (raw_imm == 12'h305) ||
    (raw_imm == 12'h341) ||
    (raw_imm == 12'h342);
*/
always @(*) begin
    ID_trap_status = `TRAP_NONE;
    ID_trapped = 1'b0;

    case (ID_opcode)
        `OPCODE_FENCE: begin // Zifencei
            if (ID_funct3 == 3'b001) begin
                ID_trapped = 1'b1;
                ID_trap_status = `TRAP_FENCEI;
            end
        end
        `OPCODE_ENVIRONMENT: begin // EBREAK, ECALL, MRET
            if (ID_funct3 == 3'b0) begin
                ID_trapped = 1'b1;
                if (raw_imm == 12'b0011_0000_0010) begin
                    ID_trap_status = `TRAP_MRET;
                end
                else if (raw_imm[0]) begin
                    ID_trap_status = `TRAP_EBREAK;
                end
                else if (raw_imm == 12'b0) begin
                    ID_trap_status = `TRAP_ECALL;
                end
            end
        end
        default: begin
            ID_trapped = 1'b0;
        end
    endcase
end

```

```

        ID_trap_status = `TRAP_NONE;
    end
end

`OPCODE_BRANCH: begin // Misaligned
if (branch_estimation == 1'b1) begin
    if (branch_target_lsbs == 2'b0) begin
        ID_trapped = 1'b0;
        ID_trap_status = `TRAP_NONE;
    end
    else begin
        ID_trapped = 1'b1;
        ID_trap_status = `TRAP_MISALIGNED_INSTRUCTION;
    end
end
end

default: begin
    ID_trapped = 1'b0;
    ID_trap_status = `TRAP_NONE;
end

endcase

EX_trapped = 1'b0;
EX_trap_status = `TRAP_NONE;

case (EX_opcode)
    `OPCODE_ENVIRONMENT: begin // EBREAK, ECALL, MRET
        if (EX_funct3 == 3'b0) begin
            EX_trapped = 1'b1;
            if (EX_raw_imm == 12'b0011_0000_0010) begin
                EX_trap_status = `TRAP_MRET;
            end
            else if (EX_raw_imm[0]) begin
                EX_trap_status = `TRAP_EBREAK;
            end
            else if (EX_raw_imm == 12'b0) begin
                EX_trap_status = `TRAP_ECALL;
            end
        end
        else begin
            EX_trapped = 1'b0;
            EX_trap_status = `TRAP_NONE;
        end
    end

    `OPCODE_STORE: begin
        case (EX_funct3)
            `STORE_SH: begin
                if (alu_result[0] == 1'b1) begin
                    EX_trapped = 1'b1;
                    EX_trap_status = `TRAP_MISALIGNED_STORE;
                end else begin

```

```

        EX_trapped = 1'b0;
        EX_trap_status = `TRAP_NONE;
    end
end
`STORE_SW: begin
    if (alu_result[1:0] != 2'b00) begin
        EX_trapped = 1'b1;
        EX_trap_status = `TRAP_MISALIGNED_STORE;
    end else begin
        EX_trapped = 1'b0;
        EX_trap_status = `TRAP_NONE;
    end
end
default: begin
    EX_trapped = 1'b0;
    EX_trap_status = `TRAP_NONE;
end
endcase
end

`OPCODE_LOAD: begin
    case (EX_func3)
        `LOAD_LH, `LOAD_LHU: begin
            if (alu_result[0] == 1'b1) begin
                EX_trapped = 1'b1;
                EX_trap_status = `TRAP_MISALIGNED_LOAD;
            end else begin
                EX_trapped = 1'b0;
                EX_trap_status = `TRAP_NONE;
            end
        end
        `LOAD_LW: begin
            if (alu_result[1:0] != 2'b00) begin
                EX_trapped = 1'b1;
                EX_trap_status = `TRAP_MISALIGNED_LOAD;
            end else begin
                EX_trapped = 1'b0;
                EX_trap_status = `TRAP_NONE;
            end
        end
        default: begin
            EX_trapped = 1'b0;
            EX_trap_status = `TRAP_NONE;
        end
    endcase
end

`OPCODE_JAL, `OPCODE_JALR: begin
    if (alu_result == 2'b0) begin
        EX_trapped = 1'b0;
        EX_trap_status = `TRAP_NONE;
    end else begin
        EX_trapped = 1'b1;
        EX_trap_status = `TRAP_MISALIGNED_INSTRUCTION;
    end
end

```

```

    end
    default: begin
        EX_trapped = 1'b0;
        EX_trap_status = `TRAP_NONE;
    end
endcase

MEM_trapped = 1'b0;
MEM_trap_status = `TRAP_NONE;

case (MEM_opcode)
    `OPCODE_STORE: begin
        case (MEM_funct3)
            `STORE_SH: begin
                if (MEM_alu_result[0] == 1'b1) begin
                    MEM_trapped = 1'b1;
                    MEM_trap_status = `TRAP_MISALIGNED_STORE;
                end else begin
                    MEM_trapped = 1'b0;
                    MEM_trap_status = `TRAP_NONE;
                end
            end
            `STORE_SW: begin
                if (MEM_alu_result[1:0] != 2'b00) begin
                    MEM_trapped = 1'b1;
                    MEM_trap_status = `TRAP_MISALIGNED_STORE;
                end else begin
                    MEM_trapped = 1'b0;
                    MEM_trap_status = `TRAP_NONE;
                end
            end
            default: begin
                MEM_trapped = 1'b0;
                MEM_trap_status = `TRAP_NONE;
            end
        endcase
    end

    `OPCODE_LOAD: begin
        case (MEM_funct3)
            `LOAD_LH, `LOAD_LHU: begin
                if (MEM_alu_result[0] == 1'b1) begin
                    MEM_trapped = 1'b1;
                    MEM_trap_status = `TRAP_MISALIGNED_LOAD;
                end else begin
                    MEM_trapped = 1'b0;
                    MEM_trap_status = `TRAP_NONE;
                end
            end
            `LOAD_LW: begin
                if (MEM_alu_result[1:0] != 2'b00) begin
                    MEM_trapped = 1'b1;
                    MEM_trap_status = `TRAP_MISALIGNED_LOAD;
                end else begin

```

```

        MEM_trapped = 1'b0;
        MEM_trap_status = `TRAP_NONE;
    end
end
default: begin
    MEM_trapped = 1'b0;
    MEM_trap_status = `TRAP_NONE;
end
endcase
end
`OPCODE_JAL, `OPCODE_JALR: begin
    if (MEM_alu_result == 2'b0) begin
        MEM_trapped = 1'b0;
        MEM_trap_status = `TRAP_NONE;
    end else begin
        MEM_trapped = 1'b1;
        MEM_trap_status = `TRAP_MISALIGNED_INSTRUCTION;
    end
end
default: begin
    MEM_trapped = 1'b0;
    MEM_trap_status = `TRAP_NONE;
end
endcase

if (MEM_trapped) begin
    trapped_combinatorial = 1'b1;
    trap_status_combinatorial = MEM_trap_status;
end else if (EX_trapped) begin
    trapped_combinatorial = 1'b1;
    trap_status_combinatorial = EX_trap_status;
end else if (ID_trapped) begin
    trapped_combinatorial = 1'b1;
    trap_status_combinatorial = ID_trap_status;
end else begin
    trapped_combinatorial = 1'b0;
    trap_status_combinatorial = `TRAP_NONE;
end
end

/* For Illegal Instruction Exception
if (csr_write_enable && csr_write && !valid_write_csr) begin
    trapped = 1'b1;
    trap_status = `TRAP_ILLEGAL_INSTRUCTION;
end
*/

// Synchronous trap signal output
always @(posedge clk or posedge reset) begin
    if (reset) begin
        trapped <= 1'b0;
        trap_status <= `TRAP_NONE;
    end else begin
        trapped <= trapped_combinatorial;
        trap_status <= trap_status_combinatorial;
    end
end

```

```

        end
    end
endmodule

```

Trap_Controller.v

```

`include "modules/headers/trap.vh"

module TrapController #(
    parameter XLEN = 32
)(
    input wire clk,
    input wire reset,
    input wire [XLEN-1:0] ID_pc,
    input wire [XLEN-1:0] EX_pc,
    input wire [XLEN-1:0] MEM_pc,
    input wire [XLEN-1:0] WB_pc,
    input wire [2:0] trap_status,    // indicates current trap type
    input wire [XLEN-1:0] csr_read_data,

    output reg [XLEN-1:0] trap_target,    // trap handler base address output
    output reg ic_clean,    // instruction cache reset signal for zifencei
    output reg debug_mode,
    output reg csr_write_enable,
    output reg [11:0] csr_trap_address,
    output reg [XLEN-1:0] csr_trap_write_data,
    output reg trap_done,    // indicates whether PTH(Pre-Trap Handling) FSM is done or not. if 0 = pc_stall
    output reg misaligned_instruction_flush,    // indicates whether the MISALIGNED INSTRUCTION pth is over and
    EX_MEM_Register should be flushed or not.
    output reg misaligned_memory_flush,
    output reg pth_done_flush,
    output reg standby_mode
);

// FSM States
localparam IDLE      = 4'b0000,
    WRITE_MEPC      = 4'b0001,
    WRITE_MCAUSE     = 4'b0010,
    READ_MTVEC       = 4'b0011,
    READ_MEPC        = 4'b0100,
    GOTO_MTVEC        = 4'b0101,
    RETURN_MRET       = 4'b0110,

    MEM_STANDBY      = 4'b0111,
    WB_STANDBY       = 4'b1000,
    RTRE_STANDBY     = 4'b1001,
    ECALL_MEPC_WRITE = 4'b1010;

// traditional FSM state architecture
reg [3:0] trap_handle_state, next_trap_handle_state;
reg debug_mode_reg;

// FSM update logic and debug_mode register update

```

```

always @(posedge clk or posedge reset) begin
    if (reset) begin
        trap_handle_state <= IDLE;
        debug_mode_reg <= 1'b0;
    end else begin
        trap_handle_state <= next_trap_handle_state;
        // debug_mode logics
        case (trap_status)
            `TRAP_MRET: begin
                if (trap_handle_state == IDLE) begin
                    debug_mode_reg <= 1'b0;
                end
            end
            `TRAP_EBREAK: begin
                if (trap_handle_state == WRITE_MCAUSE) begin
                    debug_mode_reg <= 1'b1;
                end
            end
            default: begin
                // keep debug_mode_reg
            end
        endcase
    end
end
end

```

```

// debug_mode register
always @(*) begin
    debug_mode = debug_mode_reg;
end

```

```

always @(*) begin
    // default outputs
    ic_clean          = 1'b0;
    csr_write_enable  = 1'b0;
    csr_trap_address  = 12'b0;
    csr_trap_write_data = {XLEN{1'b0}};
    trap_target       = {XLEN{1'b0}};
    trap_done         = 1'b1;
    misaligned_instruction_flush = 1'b0;
    misaligned_memory_flush = 1'b0;
    pth_done_flush     = 1'b0;
    standby_mode       = 1'b0;
    // default next state
    next_trap_handle_state = IDLE;

    case (trap_status)
        // traps that doesn't require multiple PTH FSM
        `TRAP_NONE: begin
            next_trap_handle_state = IDLE;
        end
        `TRAP_FENCEI: begin
            ic_clean = 1'b1;
            trap_done = 1'b1;
            next_trap_handle_state = IDLE;
        end
    endcase
end

```

```

end

// ----- traps that require multiple PTH FSM -----
default: begin
  case (trap_handle_state)
    IDLE: begin
      if (trap_status == `TRAP_MRET) begin
        csr_trap_address = 12'h341; //mepc
        trap_done = 1'b0;
        next_trap_handle_state = READ_MEPC;

      end else if (trap_status == `TRAP_ECALL) begin
        standby_mode = 1'b1;
        trap_done = 1'b0;
        next_trap_handle_state = MEM_STANDBY;

      end else begin
        // write current pc value to mepc CSR
        csr_write_enable = 1'b1;
        csr_trap_address = 12'h341; //mepc
        csr_trap_write_data = MEM_pc;
        trap_done = 1'b0;
        next_trap_handle_state = WRITE_MEPC;
      end
    end
  end

  MEM_STANDBY: begin
    standby_mode = 1'b1;
    trap_done = 1'b0;
    next_trap_handle_state = WB_STANDBY;
  end

  WB_STANDBY: begin
    standby_mode = 1'b1;
    trap_done = 1'b0;
    next_trap_handle_state = RTRE_STANDBY;
  end

  RTRE_STANDBY: begin
    standby_mode = 1'b1;
    trap_done = 1'b0;
    next_trap_handle_state = ECALL_MEPC_WRITE;
  end

  ECALL_MEPC_WRITE: begin
    standby_mode = 1'b0;
    // write current pc value to mepc CSR
    csr_write_enable = 1'b1;
    csr_trap_address = 12'h341; //mepc
    csr_trap_write_data = EX_pc;
    trap_done = 1'b0;
    next_trap_handle_state = WRITE_MEPC;
  end
end

```



```

WRITE_MEPC: begin
    // write mcause code value for each trap type
    csr_write_enable = 1'b1;
    csr_trap_address = 12'h342; //mcause
    if (trap_status == `TRAP_EBREAK)  csr_trap_write_data = 32'd3;
    else if (trap_status == `TRAP_ECALL)  csr_trap_write_data = 32'd11;
    else if (trap_status == `TRAP_MISALIGNED_LOAD) csr_trap_write_data = 32'd4;
    else if (trap_status == `TRAP_MISALIGNED_STORE) csr_trap_write_data = 32'd6;
    //else if (trap_status == `TRAP_ILLEGAL_INSTRUCTION) csr_trap_write_data = 32'd2;
    else csr_trap_write_data = 32'd0; // TRAP_MISALIGNED_INSTRUCTION

    trap_done = 1'b0;
    next_trap_handle_state = WRITE_MCAUSE;
end

WRITE_MCAUSE: begin
    // Enable debug mode for EBREAK and PTH escape
    if (trap_status == `TRAP_EBREAK) begin
        trap_done = 1'b1;
        next_trap_handle_state = IDLE;
    end
    else begin
        // ECALL, ILLEGAL/MISALIGNED_INSTRUCTION : read mtvec trap handler CSR value
        csr_write_enable = 1'b0;
        csr_trap_address = 12'h305; // mtvec
        trap_target = csr_read_data;
        trap_done = 1'b0;
        next_trap_handle_state = READ_MTVEC;
    end
end

READ_MTVEC: begin
    // keep mtvec value output
    csr_trap_address = 12'h305; // mtvec
    trap_target = csr_read_data;
    if (trap_status == `TRAP_MISALIGNED_INSTRUCTION) begin
        misaligned_instruction_flush = 1'b1;
    end else if (trap_status == `TRAP_MISALIGNED_STORE || trap_status == `TRAP_MISALIGNED_LOAD) begin
        misaligned_memory_flush = 1'b1;
    end
    trap_done = 1'b1;
    pth_done_flush = 1'b1;
    next_trap_handle_state = GOTO_MTVEC;
end

GOTO_MTVEC: begin
    // keep mtvec value output
    csr_trap_address = 12'h305; // mtvec
    trap_target = csr_read_data;
    if (trap_status == `TRAP_MISALIGNED_INSTRUCTION) begin
        misaligned_instruction_flush = 1'b1;
    end else if (trap_status == `TRAP_MISALIGNED_STORE || trap_status == `TRAP_MISALIGNED_LOAD) begin
        misaligned_memory_flush = 1'b1;
    end
end

```

```

        trap_done = 1'b1;
        pth_done_flush = 1'b1;
        next_trap_handle_state = IDLE;
    end

    READ_MEPC: begin
        // keep mepc value output
        csr_trap_address = 12'h341; // mepc
        trap_target = ({csr_read_data[31:2], 2'b0} + 4); // For preventing misaligned instruction address
        trap_done = 1'b0;
        next_trap_handle_state = RETURN_MRET;
    end

    RETURN_MRET: begin
        // keep mepc value output
        csr_trap_address = 12'h341; // mepc
        trap_target = ({csr_read_data[31:2], 2'b0} + 4); // For preventing misaligned instruction address
        trap_done = 1'b1;
        next_trap_handle_state = IDLE;
    end

    default: begin
        next_trap_handle_state = IDLE;
    end
endcase
end
endcase
end
endmodule

```

Control_Unit.v

```

`include "modules/headers/alu_src_select.vh"
`include "modules/headers/csr.vh"
`include "modules/headers/itype.vh"
`include "modules/headers/opcode.vh"
`include "modules/headers/rf_wd_select.vh"

module ControlUnit (
    input write_done, // signal indicating if write is done
    input trap_done, // signal indicating if Pre-Trap Handling is done
    input csr_ready,
    input IF_ID_stall,
    input [6:0] opcode, // opcode from Instruction Decoder
    input [2:0] funct3, // funct3 from Instruction Decoder

    output reg jump,
    output reg branch,
    output reg [1:0] alu_src_A_select,
    output reg [2:0] alu_src_B_select,
    output reg csr_write_enable,
    output reg register_file_write,
    output reg [2:0] register_file_write_data_select,
    output reg memory_read,

```

```

        output reg memory_write,
        output reg pc_stall
    );

    always @(*) begin
        pc_stall = (!write_done || !trap_done || !csr_ready || IF_ID_stall);

        jump = 1'b0;
        branch = 1'b0;
        alu_src_A_select = `ALU_SRC_A_NONE;
        alu_src_B_select = `ALU_SRC_B_NONE;
        csr_write_enable = 1'b0;
        register_file_write = 1'b0;
        register_file_write_data_select = `RF_WD_NONE;
        memory_read = 1'b0;
        memory_write = 1'b0;

        case (opcode)
            `OPCODE_LUI: begin // Load upper immediate
                // No jump
                jump = 0;

                // No branch
                branch = 0;

                // No alu operation
                alu_src_A_select = `ALU_SRC_A_NONE;
                alu_src_B_select = `ALU_SRC_B_NONE;

                // No CSR operation
                csr_write_enable = 0;

                // LUI instruction
                register_file_write = 1;
                register_file_write_data_select = `RF_WD_LUI;

                // No memory access
                memory_read = 0;
                memory_write = 0;
            end
            `OPCODE_AUIPC: begin
                // No jump
                jump = 0;

                // No branch
                branch = 0;

                // PC + {imm, 12'b0}
                alu_src_A_select = `ALU_SRC_A_PC;
                alu_src_B_select = `ALU_SRC_B_IMM;

                // No CSR operation
                csr_write_enable = 0;

                // Fetch added address (ALU)

```

```

        register_file_write = 1;
        register_file_write_data_select = `RF_WD_ALU;

        // No memory access
        memory_read = 0;
        memory_write = 0;
    end
    `OPCODE_JAL: begin
        // Jump
        jump = 1;

        // No branch
        branch = 0;

        // PC + {imm, 1'b0}
        alu_src_A_select = `ALU_SRC_A_PC;
        alu_src_B_select = `ALU_SRC_B_IMM;

        // No CSR operation
        csr_write_enable = 0;

        // PC+4 is saved which means jump
        register_file_write = 1;
        register_file_write_data_select = `RF_WD_JUMP;

        // No memory access
        memory_read = 0;
        memory_write = 0;
    end
    `OPCODE_JALR: begin
        // Jump
        jump = 1;

        // No branch
        branch = 0;

        // R[rs1] + imm
        alu_src_A_select = `ALU_SRC_A_RD1;
        alu_src_B_select = `ALU_SRC_B_IMM;

        // No CSR operation
        csr_write_enable = 0;

        // PC+4 is saved which means jump
        register_file_write = 1;
        register_file_write_data_select = `RF_WD_JUMP;

        // No memory access
        memory_read = 0;
        memory_write = 0;
    end
    `OPCODE_BRANCH: begin
        // No jump
        jump = 0;

```

```

// Branch
branch = 1;

// Compare R[rs1] and R[rs2]
alu_src_A_select = `ALU_SRC_A_RD1;
alu_src_B_select = `ALU_SRC_B_RD2;

// No CSR operation
csr_write_enable = 0;

// No Register File write
register_file_write = 0;
register_file_write_data_select = `RF_WD_NONE;

// No memory access
memory_read = 0;
memory_write = 0;
end
`OPCODE_LOAD: begin
// No jump
jump = 0;

// No branch
branch = 0;

// R[rs1] + imm
alu_src_A_select = `ALU_SRC_A_RD1;
alu_src_B_select = `ALU_SRC_B_IMM;

// No CSR operation
csr_write_enable = 0;

// Load instructions
register_file_write = 1;
register_file_write_data_select = `RF_WD_LOAD;

// Read from memory
memory_read = 1;
memory_write = 0;
end
`OPCODE_STORE: begin
// No jump
jump = 0;

// No branch
branch = 0;

// R[rs1] + imm
alu_src_A_select = `ALU_SRC_A_RD1;
alu_src_B_select = `ALU_SRC_B_IMM;

// No CSR operation
csr_write_enable = 0;

```

```

// No Register File write
register_file_write = 0;
register_file_write_data_select = `RF_WD_NONE;

// Write to memory
memory_read = 0;
memory_write = 1;
end
`OPCODE_ITYPE: begin
// No jump
jump = 0;

// No branch
branch = 0;

// Operation on R[rs1] and imm
alu_src_A_select = `ALU_SRC_A_RD1;

if (funct3 == `ITYPE_SRXI) begin
    alu_src_B_select = `ALU_SRC_B_SHAMT;
end
else begin
    alu_src_B_select = `ALU_SRC_B_IMM;
end

// No CSR operation
csr_write_enable = 0;

// Write alu result to Register File
register_file_write = 1;
register_file_write_data_select = `RF_WD_ALU;

// No memory access
memory_read = 0;
memory_write = 0;
end
`OPCODE_RTYPE: begin
// No jump
jump = 0;

// No branch
branch = 0;

// Operation on R[rs1] and R[rs2]
alu_src_A_select = `ALU_SRC_A_RD1;
alu_src_B_select = `ALU_SRC_B_RD2;

// No CSR operation
csr_write_enable = 0;

// Write alu result to Register File
register_file_write = 1;
register_file_write_data_select = `RF_WD_ALU;

```

```

        // No memory access
        memory_read = 0;
        memory_write = 0;
    end
    `OPCODE_FENCE: begin
        // No jump
        jump = 0;

        // No branch
        branch = 0;

        // No ALU operation
        alu_src_A_select = `ALU_SRC_A_NONE;
        alu_src_B_select = `ALU_SRC_B_NONE;

        // No CSR operation
        csr_write_enable = 0;

        // No Register File write
        register_file_write = 0;
        register_file_write_data_select = `RF_WD_NONE;

        // No memory access
        memory_read = 0;
        memory_write = 0;
    end
    `OPCODE_ENVIRONMENT: begin
        // No jump
        jump = 0;

        // No branch
        branch = 0;

        // Do CSR operation or not by funct3
        csr_write_enable = (funct3 == 0) ? 0 : 1;

        if (funct3 == 3'b0) begin
            // No alu operation
            alu_src_A_select = `ALU_SRC_A_NONE;
            alu_src_B_select = `ALU_SRC_B_NONE;

            // No Register File write
            register_file_write = 0;
            register_file_write_data_select = `RF_WD_NONE;
        end
        else begin
            if (funct3 == `CSR_CSRRW || funct3 == `CSR_CSRRWI) begin
                alu_src_B_select = `ALU_SRC_B_NONE;
            end
            else begin
                alu_src_B_select = `ALU_SRC_B_CSR; // src_B is CSR value
            end
        end
    end

```

```

        if (funct3 == `CSR_CSRRW || funct3 == `CSR_CSRRS || funct3 ==
`CSR_CSRRC) begin
            alu_src_A_select = `ALU_SRC_A_RD1; // non-immediate CSR
instructions require src_A to be rd1
        end
        else begin
            alu_src_A_select = `ALU_SRC_A_RS1; // immediate CSR
instructions require src_A to be rs1
        end

        // Write CSR value to Register File
        register_file_write = 1;
        register_file_write_data_select = `RF_WD_CSR;
    end

    // No memory access
    memory_read = 0;
    memory_write = 0;
end
default: begin
    jump = 1'b0;
    branch = 1'b0;
    alu_src_A_select = `ALU_SRC_A_NONE;
    alu_src_B_select = `ALU_SRC_B_NONE;
    csr_write_enable = 1'b0;
    register_file_write = 1'b0;
    register_file_write_data_select = `RF_WD_NONE;
    memory_read = 1'b0;
    memory_write = 1'b0;
end
endcase
end

endmodule
ALU.v

`include "modules/headers/alu_op.vh"

module ALU (
    input [31:0] src_A,        // source operand A
    input [31:0] src_B,        // source operand B
    input [3:0] alu_op,        // ALU operation signal (from ALU Control module)

    output reg [31:0] alu_result, // ALU result
    output reg alu_zero          // zero flag
);

always @(*) begin
    case (alu_op)
        `ALU_OP_ADD: begin
            alu_result = src_A + src_B;
        end

        `ALU_OP_SUB: begin

```



```

        alu_result = src_A - src_B;
    end

    `ALU_OP_AND: begin
        alu_result = src_A & src_B;
    end

    `ALU_OP_OR: begin
        alu_result = src_A | src_B;
    end

    `ALU_OP_XOR: begin
        alu_result = src_A ^ src_B;
    end

    `ALU_OP_SLT: begin
        alu_result = ($signed(src_A) < $signed(src_B)) ? 32'd1 : 32'd0;
    end

    `ALU_OP_SLTU: begin
        alu_result = (src_A < src_B) ? 32'd1 : 32'd0;
    end

    `ALU_OP_SLL: begin
        alu_result = src_A << src_B;
    end

    `ALU_OP_SRL: begin
        alu_result = src_A >> src_B;
    end

    `ALU_OP_SRA: begin
        alu_result = $signed(src_A) >>> src_B;
    end

    `ALU_OP_ABJ: begin
        alu_result = src_B & (~src_A);
    end

    `ALU_OP_BPA: begin
        alu_result = src_A;
    end

    `ALU_OP_NOP: begin
        alu_result = 32'd0;
    end

    default: begin
        alu_result = 32'd0; // Default case: zero result
    end
endcase

alu_zero = (alu_result == 32'd0); // Zero flag: set if result is zero
end

```

```
endmodule
```

ALU_Controller.v

```
`include "modules/headers/alu_op.vh"
`include "modules/headers/branch.vh"
`include "modules/headers/csr.vh"
`include "modules/headers/itype.vh"
`include "modules/headers/opcode.vh"
`include "modules/headers/rtype.vh"

module ALUController (
    input [6:0] opcode,          // opcode
        input [2:0] funct3,          // funct3
        input funct7_5,          // 5th index of funct7 (starting from 0th index)
        input imm_10,          // 10th index of imm (starting from 0th index)

    output reg [3:0] alu_op      // ALU operation signal
);

always @(*) begin
    case (opcode)

        `OPCODE_AUIPC: begin
            alu_op = `ALU_OP_ADD;
        end
        `OPCODE_JAL: begin
            alu_op = `ALU_OP_ADD;
        end
        `OPCODE_JALR: begin
            alu_op = `ALU_OP_ADD; // JALR instruction requires addition
        end
        `OPCODE_BRANCH: begin
            case (funct3)
                `BRANCH_BEQ: begin
                    alu_op = `ALU_OP_SUB; // If subtraction result is zero, equal
                end
                `BRANCH_BNE: begin
                    alu_op = `ALU_OP_SUB; // If subtraction result is not zero, not
equal
                end
                `BRANCH_BLT: begin
                    alu_op = `ALU_OP_SLT; // If SLT result is not zero, less
                end
                `BRANCH_BGE: begin
                    alu_op = `ALU_OP_SLT; // If SLT result is zero, greater or equal
                end
                `BRANCH_BLTU: begin
                    alu_op = `ALU_OP_SLTU; // If SLTU result is not zero, less
(unsigned)
                end
                `BRANCH_BGEU: begin
                    alu_op = `ALU_OP_SLTU; // If SLTU result is zero, greater or equal
(unsigned)
                end
            end
        end
    end
end
```

```

        end
        default: begin
            alu_op = `ALU_OP_NOP;
        end
    endcase
end
`OPCODE_LOAD: begin
    alu_op = `ALU_OP_ADD; // Every load instruction requires addition
end
`OPCODE_STORE: begin
    alu_op = `ALU_OP_ADD; // Every store instruction requires addition
end
`OPCODE_ITYPE: begin
    case (funct3)
        `ITYPE_ADDI: begin
            alu_op = `ALU_OP_ADD;
        end
        `ITYPE_SLLI: begin
            alu_op = `ALU_OP_SLL;
        end
        `ITYPE_SLTI: begin
            alu_op = `ALU_OP_SLT;
        end
        `ITYPE_SLTIU: begin
            alu_op = `ALU_OP_SLTU;
        end
        `ITYPE_XORI: begin
            alu_op = `ALU_OP_XOR;
        end
        `ITYPE_SRXI: begin // srli or srai
            if (imm_10) begin
                alu_op = `ALU_OP_SRA; // srai : imm[10] = 1
            end
            else begin
                alu_op = `ALU_OP_SRL; // srli : imm[10] = 0
            end
        end
        `ITYPE_ORI: begin
            alu_op = `ALU_OP_OR; // ori : 110 ; -
        end
        `ITYPE_ANDI: begin
            alu_op = `ALU_OP_AND; // andi : 111 ; -
        end
        default: begin
            alu_op = `ALU_OP_NOP;
        end
    endcase
end
`OPCODE_RTYPE: begin
    case (funct3)
        `RTYPE_ADDSUB: begin // add or sub
            if (funct7_5) begin
                alu_op = `ALU_OP_SUB; // sub : funct7 = 0100000
            end
        end
    end
end

```

```

        else begin
            alu_op = `ALU_OP_ADD; // add : funct7 = 0000000
        end
    end
`RTYPE_SLL: begin
    alu_op = `ALU_OP_SLL;
end
`RTYPE_SLT: begin
    alu_op = `ALU_OP_SLT;
end
`RTYPE_SLTU: begin
    alu_op = `ALU_OP_SLTU;
end
`RTYPE_XOR: begin
    alu_op = `ALU_OP_XOR;
end
`RTYPE_SR: begin // srl or sra
    if (funct7_5) begin
        alu_op = `ALU_OP_SRA; // sra : funct7 = 0100000
    end
    else begin
        alu_op = `ALU_OP_SRL; // srl : funct7 = 0000000
    end
end
`RTYPE_OR: begin
    alu_op = `ALU_OP_OR;
end
`RTYPE_AND: begin
    alu_op = `ALU_OP_AND;
end
default: begin
    alu_op = `ALU_OP_NOP;
end
endcase
end

`OPCODE_ENVIRONMENT: begin
    case (funct3)
        `CSR_CSRRW: begin
            alu_op = `ALU_OP_BPA;
        end
        `CSR_CSRRS: begin
            alu_op = `ALU_OP_OR;
        end
        `CSR_CSRRC: begin
            alu_op = `ALU_OP_ABJ;
        end
        `CSR_CSRRWI: begin
            alu_op = `ALU_OP_BPA;
        end
        `CSR_CSRRSI: begin
            alu_op = `ALU_OP_OR;
        end
        `CSR_CSRRCI: begin
            alu_op = `ALU_OP_ABJ;
        end
    end
end

```

```

                                end
                                default: begin
                                    alu_op = `ALU_OP_NOP;
                                end
                            endcase
                        end
                        default: begin
                            alu_op = `ALU_OP_NOP;
                        end
                    endcase
                end
            end
        endcase
    end

endmodule

```

Branch_Logic.v

```

`include "modules/headers/branch.vh"

module BranchLogic (
    input branch,
    input branch_estimation,
    input alu_zero,
    input [2:0] funct3,
    input [31:0] pc,
    input [31:0] imm,

    output reg branch_taken,
    output reg [31:0] branch_target_actual,
    output reg branch_prediction_miss
);

always @(*) begin
    if (branch) begin
        case (funct3)
            `BRANCH_BEQ: branch_taken = alu_zero;
            `BRANCH_BNE: branch_taken = ~alu_zero;
            `BRANCH_BLT: branch_taken = ~alu_zero;
            `BRANCH_BGE: branch_taken = alu_zero;
            `BRANCH_BLTU: branch_taken = ~alu_zero;
            `BRANCH_BGEU: branch_taken = alu_zero;
            default:    branch_taken = 1'b0;
        endcase

        branch_prediction_miss = (branch_estimation != branch_taken);

        if (branch_taken) begin
            branch_target_actual = pc + imm;
        end else begin
            branch_target_actual = pc + 4;
        end
    end
    else begin
        branch_taken = 1'b0;
        branch_target_actual = 32'b0;
    end
end

```

```

        branch_prediction_miss = 1'b0;
    end
end

endmodule

```

EX_MEM_Register.v

```

module EX_MEM_Register #(
    parameter XLEN = 32
)(
    // pipeline register control signals
    input wire clk,
    input wire reset,
    input wire flush,
    input wire EX_MEM_stall,

    // signals from ID/EX register
    input wire [XLEN-1:0] EX_pc, // for debugging
    input wire [XLEN-1:0] EX_pc_plus_4,
    input wire [31:0] EX_instruction,

    input wire EX_memory_read,
    input wire EX_memory_write,
    input wire [2:0] EX_register_file_write_data_select,
    input wire EX_register_write_enable,
    input wire EX_csr_write_enable,
    input wire [6:0] EX_opcode,
    input wire [2:0] EX_funct3,
    input wire [4:0] EX_rs1,
    input wire [4:0] EX_rd,
    input wire [XLEN-1:0] EX_read_data2, // Register File to Data Memory read data
    input wire [XLEN-1:0] EX_imm,
    input wire [19:0] EX_raw_imm,
    input wire [XLEN-1:0] EX_csr_read_data,

    // signals from EX phase
    input wire [XLEN-1:0] EX_alu_result,

    // signals to EX/MEM register
    output reg [XLEN-1:0] MEM_pc,
    output reg [XLEN-1:0] MEM_pc_plus_4,
    output reg [31:0] MEM_instruction,

    output reg MEM_memory_read,
    output reg MEM_memory_write,
    output reg [2:0] MEM_register_file_write_data_select,
    output reg MEM_register_write_enable,
    output reg MEM_csr_write_enable,
    output reg [6:0] MEM_opcode,
    output reg [2:0] MEM_funct3,
    output reg [4:0] MEM_rs1,
    output reg [4:0] MEM_rd,
    output reg [XLEN-1:0] MEM_read_data2,

```

```

output reg [XLEN-1:0] MEM_imm,
output reg [19:0] MEM_raw_imm,
output reg [XLEN-1:0] MEM_csr_read_data,

output reg [XLEN-1:0] MEM_alu_result
);

always @(posedge clk or posedge reset) begin
  if (reset) begin
    MEM_pc <= {XLEN{1'b0}};
    MEM_pc_plus_4 <= {XLEN{1'b0}};
    MEM_instruction <= 32'h0000_0013; // ADDI x0, x0, 0 = RISC-V NOP, HINT

    MEM_memory_read <= 1'b0;
    MEM_memory_write <= 1'b0;
    MEM_register_file_write_data_select <= 3'b0;
    MEM_register_write_enable <= 1'b0;
    MEM_csr_write_enable <= 1'b0;
    MEM_opcode <= 7'b0;
    MEM_funct3 <= 3'b0;
    MEM_rs1 <= 5'b0;
    MEM_rd <= 5'b0;
    MEM_read_data2 <= {XLEN{1'b0}};
    MEM_imm <= {XLEN{1'b0}};
    MEM_raw_imm <= 20'b0;
    MEM_csr_read_data <= {XLEN{1'b0}};

    MEM_alu_result <= {XLEN{1'b0}};
  end else begin
    if (flush) begin
      MEM_pc <= {XLEN{1'b0}};
      MEM_pc_plus_4 <= {XLEN{1'b0}};
      MEM_instruction <= 32'h0000_0013; // ADDI x0, x0, 0 = RISC-V NOP, HINT

      MEM_memory_read <= 1'b0;
      MEM_memory_write <= 1'b0;
      MEM_register_file_write_data_select <= 3'b0;
      MEM_register_write_enable <= 1'b0;
      MEM_csr_write_enable <= 1'b0;
      MEM_opcode <= 7'b0;
      MEM_funct3 <= 3'b0;
      MEM_rs1 <= 5'b0;
      MEM_rd <= 5'b0;
      MEM_read_data2 <= {XLEN{1'b0}};
      MEM_imm <= {XLEN{1'b0}};
      MEM_raw_imm <= 20'b0;
      MEM_csr_read_data <= {XLEN{1'b0}};

      MEM_alu_result <= {XLEN{1'b0}};
    end else if (!EX_MEM_stall) begin
      MEM_pc <= EX_pc;
      MEM_pc_plus_4 <= EX_pc_plus_4;
      MEM_instruction <= EX_instruction;
    end
  end
end

```

```
MEM_memory_read <= EX_memory_read;
MEM_memory_write <= EX_memory_write;
MEM_register_file_write_data_select <= EX_register_file_write_data_select;
MEM_register_write_enable <= EX_register_write_enable;
MEM_csr_write_enable <= EX_csr_write_enable;
MEM_opcode <= EX_opcode;
MEM_funct3 <= EX_funct3;
MEM_rs1 <= EX_rs1;
MEM_rd <= EX_rd;
MEM_read_data2 <= EX_read_data2;
MEM_imm <= EX_imm;
MEM_raw_imm <= EX_raw_imm;
MEM_csr_read_data <= EX_csr_read_data;

MEM_alu_result <= EX_alu_result;
end
end

end

endmodule
```